PG -TABNET: A HYBRID PHYSICS-GUIDED DEEP TABULAR FRAMEWORK FOR ELECTRIC VEHICLE ENERGY CONSUMPTION FORECASTING

a project report submitted by

FINNEY RAJAN S (REG. NO: URK22CS1056)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

under the supervision of

Dr. S. Stewart Kirubakaran M.Tech., (Ph.D.)

Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved; NAAC Accredited A++
Karunya Nagar, Coimbatore - 641 114, Tamil Nadu, India.

DIVISION OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

KARUNYA INSTITUTE OF TECHNOLOGY AND SCIENCES

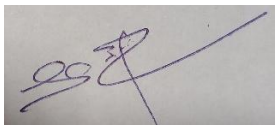
(Declared as Deemed-to-be-University under Sec-3 of the UGC Act, 1956)

Karunya Nagar, Coimbatore - 641 114, India.

APRIL 2026

BONAFIDE CERTIFICATE

Certified that this project report **"PG -TABNET: A HYBRID PHYSICS-GUIDED DEEP TABULAR FRAMEWORK FOR ELECTRIC VEHICLE ENERGY CONSUMPTION FORECASTING"** is the bonafide work of "FINNEY RAJAN S (REG. NO: URK22CS1056)**"** who carried out the project work under my supervision.



SIGNATURE

Dr. S. Stewart Kirubakaran

Supervisor
Assistant Professor
Division of Computer Science and Engineering

SIGNATURE

Dr. G. Jaspher W Kathrine

Head of the Division
Division of Computer Science and Engineering

.

Submitted for the Project Viva Voce held on 20-04-2026.

Examiner

ACKNOWLEDGEMENT

First and foremost, I praise and thank **ALMIGHTY GOD** for giving me the will power and confidence to carry out my project.

I am grateful to our beloved founders Late **Dr. D.G.S. Dhinakaran**, **C.A.I.I.B, Ph.D.,** and **Dr. Paul Dhinakaran**, **M.B.A., Ph.D.,** for their love and always remembering me in their prayers.

I extend my thanks to **Dr. R. Elijah Blessing, Ph.D.,** our honorable Vice Chancellor, and **Dr. S. J. Vijay, Ph.D.,** our respected Registrar for giving me the opportunity to carry out this project.

I would like to place my heart-felt thanks and gratitude to **Dr. Jaspher W Kathrine, Ph.D.,** HOD, Division of Computer Science and Engineering for her encouragement and guidance.

I am grateful to my guide, **Dr. S. Stewart Kirubakaran, M.Tech., (Ph.D.,)** Assistant Professor, Division of Computer Science and Engineering for his valuable support, advice and encouragement.

I also thank all the staff members of the Division of Computer Science and Engineering for extending their helping hands to make this project work a success.

I would also like to thank all my friends and my parents who have prayed and helped me during the project work.

ABSTRACT

Accurate prediction of energy consumption in electric vehicles (EVs) plays a crucial role in battery management systems, route planning, charging scheduling, and mitigation of range anxiety. The worldwide increase in EV adoption requires stakeholders to find dependable energy forecasting models that function effectively under actual driving conditions with increasing urgency. The amount of energy used by EVs depends on multiple factors that include how the vehicle operates and how drivers behave and the type of roads and current traffic patterns and the prevailing weather conditions. The traditional physics-based models provide understandable results because they use vehicle dynamics as their base, yet their use of basic driving scenarios requires particular system settings that restrict their ability to operate in different driving situations. Data-driven machine learning models achieve high prediction accuracy, but their results lack physical explanation, zand their models produce physically impossible outcomes while showing unreliable results under two different conditions.

The project implements PG-TabNet as a solution to multiple problems through its development of a Physics-Guided Tabular Network framework to forecast electric vehicle energy consumption. The new method combines physics-based vehicle dynamics modeling with deep learning methods which the system uses for processing structured tabular data. The framework does not predict total energy consumption but calculates theoretical energy consumption through fundamental physical principles which include resistive forces and traction power and drivetrain efficiency. A deep learning model based on the TabNet architecture is then employed to learn the residual energy component which represents the deviation between the measured energy consumption and the physics-based theoretical estimate. The model uses residual learning to capture nonlinear driver behavior and traffic disturbances and auxiliary loads and environmental uncertainties which physical equations cannot fully explain.

The training process uses a physics-guided loss function to maintain physical consistency and model robustness by penalizing the system for exceeding the allowed range of energy theoretical baseline values. The introduction of an auxiliary prediction task for battery State-of-Charge (SOC) drop serves to enhance representation learning capabilities while improving generalization performance. The evaluation process for the model uses an electric vehicle dataset which meets physics requirements and includes controlled noise elements to replicate actual uncertainty conditions. The experimental results show that the PG-TabNet framework outperforms the baseline TabNet model under noisy operating conditions because it achieves better prediction accuracy and maintains operational stability.

The project results demonstrate that adding domain expertise through physics-guided residual learning improves energy consumption prediction results by making them more accurate and easier to interpret and more stable. The proposed framework provides a practical solution for EV energy management systems which can operate in real-world environments while establishing a research foundation for developing physics-guided machine learning techniques in intelligent transportation systems.

TABLE OF CONTENTS

CHAPTER/TITLE	PAGE NO.
ABSTRACT	**IV**
LIST OF FIGURES	**VI**
LIST OF TABLES	**VII**
LIST OF SYMBOLS AND ABBREVATION	**VIII**
CHAPTER 1: INTRODUCTION	**1**
1.1 Objective	**2**
1.2 Problem Statement	**3**
1.3 Chapter Wise Summary	**4**
CHAPTER 2: SYSTEM ANALYSIS	**5**
2.1 Existing System	**5**
2.2 Proposed System	**7**
2.3 Use Case Analysis	**8**
2.4 Requirement Specification	**9**
2.4.1 Functional Requirements	**9**
2.4.2 Non-Functional Requirements	**10**
2.4.3 Hardware Requirements	**10**
2.4.4 Software Requirements	**11**
CHAPTER 3: SYSTEM DESIGN	**12**
3.1 Detailed Design (Architecture Diagram and Description)	**12**
3.2 Design of Methodology	**14**
3.3 Physics-Based Energy Modeling	**16**
3.4 Modules	**18**
3.5 Database Design	**20**
3.5.1 Entity Relationship Diagram	**20**
3.5.2 Unified Dataset Table	**22**
CHAPTER 4: SYSTEM IMPLEMENTATION	**23**
4.1 Module Implementation	**23**
4.2 Testing	**26**
CHAPTER 5: CONCLUSION AND FUTURE SCOPE	**30**
5.1 Conclusion	**30**
5.2 Future Scope	**30**
Appendix A: Source Code	**32**
Appendix B: Screenshots	**44**
REFERENCES	**46**

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE
Fig 3.1	Architecture Diagram – Proposed PG-TabNet Framework	**12**
Fig 3.2	Methodology Flow Diagram	**14**
Fig 3.3	Forces acting on an electric vehicle during motion.	**16**
Fig 3.4	Entity Relationship Diagram	**21**
Fig 3.5	Correlation Heatmap Analysis of dataset	**22**
Fig 4.1	Residual Comparison of Base vs Proposed Model	**25**
Fig 4.2	Actual vs Predicted Energy Comparison	**26**
Fig 4.3	Noise Robustness Ablation Comparison	**28**
Fig 4.4	Training and validation loss convergence Comparison	**29**

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 2.1	Use Case Table	**9**
Table 3.1	Dataset Description Table	**22**
Table 4.1	Performance Metrics Comparison	**28**

LIST OF SYMBOLS AND ABBREVIATIONS

A. List of Symbols:

Symbol	Description	Unit
v	Vehicle speed	m/s
a	Longitudinal acceleration	m/s²
m	Vehicle mass	kg
g	Gravitational acceleration	m/s²
θ	Road inclination angle	radians
C_r	Rolling resistance coefficient	—
ρ	Air density	kg/m³
C_d	Aerodynamic drag coefficient	—
A_f	Vehicle frontal area	m²
F_r	Rolling resistance force	N
F_a	Aerodynamic drag force	N
F_g	Gravitational force	N
F_i	Inertial resistance force	N
F_{total}	Total resistive force	N
P_{trac}	Traction power	kW
η	Drivetrain efficiency	—
P_{elec}	Electrical power demand	kW
E_{theory}	Theoretical energy consumption	kWh
E_{actual}	Measured energy consumption	kWh
E_{res}	Residual energy	kWh
λ	Physics regularization coefficient	—
γ	SOC loss weighting coefficient	—

B. List of Abbreviations:

Abbreviation	Full Form
EV	Electric Vehicle
PG-TabNet	Physics-Guided Tabular Network
SOC	State of Charge
ML	Machine Learning
DL	Deep Learning
AI	Artificial Intelligence
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
R^2	Coefficient of Determination
CSV	Comma-Separated Values
GPU	Graphics Processing Unit
ER Diagram	Entity Relationship Diagram
MSE	Mean Squared Error
kWh	Kilowatt-hour
km/h	Kilometer per hour
m/s	Meter per second
m/s²	Meter per second squared

CHAPTER 1

INTRODUCTION

The global transportation sector is undergoing a significant transition toward electric mobility due to increasing concerns over fossil fuel depletion and environmental pollution and climate change. Electric vehicles (EVs) have emerged as a sustainable alternative to conventional internal combustion engine vehicles which produce fewer emissions and operate with higher energy efficiency. EV adoption has created the need for energy management systems which ensure vehicle functionality and battery performance and customer satisfaction.

Electric vehicle operation faces its main obstacle when it comes to estimating energy usage throughout its complete operational time. Reliable energy estimation becomes vital for battery management systems and route planning and charging scheduling and range anxiety reduction in users. EVs can only use battery energy which operates under conditions as their main power source. Multiple factors interact to determine EV energy use which includes vehicle speed and acceleration and road gradient and vehicle mass and traffic conditions and driving patterns and environmental factors such as wind and weather conditions. The nonlinear and dynamic nature of these factors makes accurate energy forecasting a complex task particularly under real-world driving scenarios.

Traditional approaches to EV energy consumption prediction are largely based on physics-based vehicle dynamics models. The systems estimate energy requirements by determining which resistive forces distance from rolling resistance and aerodynamic drag and road slope gravitational force and road slope and inertial effects. The scientific models provide two advantages because they allow people to understand their functioning and prevent any deviation from basic scientific principles. The scientific models still depend on basic scientific rules but their ability to adapt to real-world situations suffers because they depend on fixed parameters and basic scientific rules. The system loses its ability to make accurate predictions when driving patterns and traffic conditions and environmental factors vary widely.

Academics have studied data-driven machine learning methods as a solution to these problems based on their effectiveness in multiple research studies. Historical driving data serves as the learning material from which machine learning models develop patterns which enable them to understand how different input features affect energy consumption. The models deliver accurate predictions, yet they don't provide physical insights which lead to generating predictions that conflict with physical laws. The data-driven models 10have results which depend on yet-to-be-seen operational environments and noisy data which harms their effectiveness as energy management systems for essential safety operations.

The research program combines physics-based and data-driven techniques as its core approach through the development of a Physics-Guided Tabular Network PG-TabNet framework built to forecast EV energy consumption. The research method combines physics-based energy modeling with deep learning techniques which function optimally for working with structured tabular data. The basic vehicle dynamics equations allow

researchers to create their first theoretical energy consumption estimate. Residual energy makes up the difference between total measured energy and theoretical energy values which the learning model uses to make predictions. The model uses residual learning to understand how driving behavior and traffic differences and auxiliary loads and environmental changes bring about nonlinear outcomes.

The physics-guided loss function works during training to maintain physical consistency by applying penalties for all energy deviations from the theoretical energy baseline. The creation of an auxiliary prediction task for battery SOC drop provides extra features which assist learning through better representation. The proposed PG-TabNet framework achieves better accuracy and model stability compared to baseline models through its evaluation on a physics-consistent electric vehicle dataset which includes both clean and noise-injected conditions.

The research program intends to create an energy prediction framework for electric vehicles which meets three requirements: it must deliver precise results while providing understandable information and maintaining strong performance in actual operating conditions. The proposed approach enables trustworthy intelligent transportation systems and electric vehicle energy management solutions through the combination of physical knowledge and deep tabular learning.

1.1 Objective

The primary objective of this project is to design and develop an accurate, robust, and physically interpretable framework for predicting electric vehicle (EV) energy consumption under real-world driving conditions. The project aims to overcome the limitations of conventional physics-based models and purely data-driven machine learning approaches by integrating domain knowledge from vehicle dynamics with deep learning techniques for structured tabular data.

The proposed work establishes a physics-guided learning framework in which researchers first calculate theoretical energy consumption through fundamental vehicle dynamics equations. The model uses residual energy learning to predict energy consumption because it focuses on learning the difference between actual energy usage and energy usage predicted by the physics-based model. The residual learning strategy helps to reduce model complexity while the learning algorithm can better capture nonlinear driver behavior patterns and traffic pattern changes and auxiliary power consumption and environmental disturbances that occur during operation.

The prediction process needs physical consistency and prediction robustness for successful implementation. The physics-guided loss function training process uses training data to punish large energy deviations which helps the model create energy predictions that adhere to physical laws. The introduction of SOC battery drop prediction through an auxiliary task helps to create better feature representation which leads to better model generalization and training process stabilization.

The project tests the proposed Physics-Guided Tabular Network (PG-TabNet) framework by performing extensive testing to determine its effectiveness. The assessment includes three steps which require researchers to compare their model to a base TabNet model and measure prediction accuracy using established assessment standards and they must evaluate system stability in noisy conditions that represent actual operational uncertainty.

The project aims to create a trustworthy EV energy prediction system that produces interpretable results which will enable intelligent energy management and route planning and decision-making processes in electric vehicle operations while serving as a solid base for future research work in physics-guided machine learning.

1.2 Problem Statement

The prediction process for energy consumption in electric vehicles (EVs) encounters significant obstacles because real-world driving conditions display both complex characteristics and changing patterns. The energy consumption of EVs depends on a network of factors that include vehicle speed, acceleration, vehicle mass, road gradient, traffic conditions, environmental effects, and driving behavior. The factors exhibit nonlinear interactions, which complicate the creation of models that can accurately predict outcomes, while also providing reliable results throughout various operating conditions.

The traditional energy consumption models based on physics use vehicle dynamics principles to estimate electric vehicle energy requirements by applying rolling resistance and aerodynamic drag and gravitational force and inertial effects. The models maintain physical consistency with essential laws, yet they depend on generalizations through predetermined parameters. The models exhibit decreased predictive capability across diverse driving conditions and traffic disruptions and auxiliary load applications and environmental variations. The models establish boundaries for operational implementation, particularly in environments that experience continuous condition shifts.

The development of prediction accuracy through machine learning requires data-driven model systems that study historical driving data to uncover complex nonlinear relationships. The models fail to provide physical interpretation, which leads to prediction outcomes that do not conform to the rules of physics. Data-driven systems experience prediction failures due to their vulnerability to noisy measurements and unrecognized driving situations and dataset bias, which makes the systems unsuitable for safety-critical applications like vehicle energy management and range estimation.

The absence of physical constraints in many machine learning models leads to decreased trust and Explainability which prevents their real-world application in electric vehicle platforms. The system requires a prediction framework that links physics-based modeling with data-driven learning while resolving their individual weaknesses.

The development of an energy consumption prediction model for electric vehicles serves as the primary objective of this project which encompasses the following requirements:

- Maintains physical consistency and interpretability,
- Accurately captures nonlinear and uncertain real-world driving effects,

- Remains robust under noisy and variable operating conditions, and
- Is suitable for structured tabular datasets commonly available in EV telemetry systems.

The research problem receives resolution through the introduction of Physics-Guided Tabular Network (PG-TabNet) which combines physics-based theoretical energy modeling with deep learning-based residual learning to deliver precise and dependable energy consumption prediction results for electric vehicles.

1.3 Chapter Wise Summary

This project report is organized into five chapters, each describing a specific aspect of the proposed work in a structured and systematic manner.

Chapter 1 – Introduction presents the background and motivation for electric vehicle energy consumption prediction. It discusses the importance of accurate energy forecasting, highlights the limitations of existing physics-based and data-driven approaches, and introduces the proposed Physics-Guided Tabular Network (PG-TabNet) framework. The chapter also outlines the objectives and problem statement of the project.

Chapter 2 – System Analysis examines the existing approaches used for EV energy consumption prediction and identifies their limitations. It then introduces the proposed system and explains how it addresses the identified challenges. This chapter also includes use case analysis and a detailed requirement specification, covering functional, non-functional, hardware, and software requirements.

Chapter 3 – System Design describes the overall architecture and detailed design of the proposed PG-TabNet framework. It explains the methodology adopted for physics-based energy modeling and residual learning, outlines the system modules, and presents relevant design diagrams such as the architecture diagram and entity relationship diagram as required.

Chapter 4 – System Implementation details the implementation of the proposed system. It discusses module-level implementation, data preprocessing, model training, and evaluation procedures. This chapter also covers testing strategies employed to validate the correctness and performance of the system.

Chapter 5 – Conclusion and Future Scope summarizes the work carried out in the project and discusses the key findings and contributions. It also outlines potential directions for future enhancements, including model extensions, scalability, and real-world deployment considerations.

CHAPTER 2

SYSTEM ANALYSIS

System analysis is a critical phase in the development of any software or intelligent system, as it focuses on understanding the problem domain, evaluating existing solutions, and defining the requirements for the proposed system. In the context of electric vehicle (EV) energy consumption prediction, system analysis helps identify the limitations of current modeling approaches and establishes the need for a more accurate, robust, and physically consistent prediction framework.

Energy consumption prediction systems for EVs must operate under highly dynamic and uncertain conditions. Variations in driving behavior, road topology, traffic patterns, and environmental factors introduce significant complexity into the prediction task. Existing solutions attempt to address this complexity either through physics-based vehicle dynamics models or through data-driven machine learning techniques. However, each of these approaches exhibits notable shortcomings when applied independently, particularly in real-world deployment scenarios.

This chapter presents a systematic analysis of the existing EV energy prediction systems and highlights their limitations. Based on this analysis, the proposed Physics-Guided Tabular Network (PG-TabNet) system is introduced, emphasizing how it addresses the identified gaps. The chapter further includes use case analysis and a detailed requirement specification, covering functional, non-functional, hardware, and software requirements necessary for implementing the proposed system effectively.

2. 1 Existing System

Existing systems for electric vehicle (EV) energy consumption prediction can be broadly classified into two main categories: physics-based modeling approaches and data-driven machine learning approaches. These systems aim to estimate the energy required for a vehicle to complete a driving segment or trip based on available vehicle, road, and environmental parameters. While each category offers certain advantages, both exhibit significant limitations when applied independently in real-world conditions.

Physics-Based Energy Prediction Systems

The physics-based EV energy prediction systems use classical vehicle dynamics and energy balance principles as their fundamental scientific foundation. The model calculates traction power and energy consumption by measuring resistive forces which include rolling resistance and aerodynamic drag and road gradient gravitational force and acceleration inertial force. The system calculates power usage through total power demand which it tracks over time to determine energy consumption. Researchers employ these systems extensively because their results are easy to understand, and their operations comply with established scientific principles.

The physics-based systems deliver clear predictions which maintain physical accuracy through their fundamental system design. The system requires constant rolling resistance coefficients and drivetrain efficiency and idealized environmental conditions as essential fixed parameters for their operation. The system operates through significant factors which include driver aggressiveness and traffic congestion and auxiliary power usage and battery aging and real-time environmental variability. The systems cannot perform correctly across various driving conditions because they experience high operational failures in actual driving situations.

Data-Driven Machine Learning Systems

The development of data-driven machine learning systems enables EV energy prediction through their ability to bypass the limitations which physics-based models impose. The system discovers statistical connections between its input features and energy consumption through analysis of historical driving patterns. The models which researchers commonly use include linear regression and decision trees and ensemble methods and deep learning architectures. The system uses extensive datasets to create highly accurate models which capture complex nonlinear relationships which physical equations cannot effectively represent.

The data-driven systems which provide better predictive results encounter multiple issues which prevent their effective use. The system fails to deliver physical interpretability. The system fails to deliver physical interpretability. The machine learning models generate results which lack physical validity in most situations when users apply the models to new driving situations or random sensor errors. The system requires data quality and distribution to function correctly because its operations break down when users operate it outside the situations shown in its training data.

Limitations of Existing Systems

The data-driven and physics-based systems which predict EV energy consumption face fundamental restrictions which affect their practical performance in actual situations. The data-driven systems operate without physical consistency and their performance suffers from real-time variability which affects their results. The existing methods treat energy prediction as a single-task operation. The methods overlook auxiliary physical indicators which include battery State-of-Charge (SOC) dynamics that could improve learning stability.

The current systems operate without the capacity to process unpredictable elements which stem from real-world conditions. The reliability of predictions suffers when sensor errors occur and measurement noise enters the system and environmental disturbances affect the prediction process. The current systems fail to achieve the three essential requirements which EV energy management applications need for accurate energy prediction and robust performance and clear understanding of their results.

The existing systems face challenges which require the development of a hybrid framework which combines physical knowledge with data-driven learning. The system needs to adhere to physical laws while using

machine learning capabilities to model all actual world patterns. The following section introduces the proposed system.

2. 2 Proposed System

The proposed system introduces a Physics-Guided Tabular Network (PG-TabNet) framework for predicting electric vehicle (EV) energy consumption in a physically consistent, accurate, and robust manner. The system uses vehicle dynamics knowledge and deep learning techniques that work with structured tabular data to solve the limitations of current physics-based and data-driven systems.

The proposed system uses physics-based vehicle dynamics to calculate theoretical energy estimation, which uses deep neural networks for residual learning. The system first establishes a theoretical energy consumption estimate based on fundamental physical principles, which include resistive forces and traction power and drivetrain efficiency, before making predictions about total energy consumption. The theoretical energy serves as a baseline which establishes the main patterns of energy consumption.

The proposed system uses the TabNet architecture, which was developed for tabular datasets, as its deep learning component. The TabNet model is trained to predict the residual energy, which is the difference between the actual measured energy consumption and the physics-based theoretical energy estimate. The model learns only the residual component to capture driver behavior and traffic variability and auxiliary loads and environmental disturbances that exist beyond the physics equations.

The proposed system uses a physics-guided loss function during training to maintain physical consistency for its prediction outputs. The loss function uses predicted energy results to calculate the prediction accuracy between actual energy values and theoretical energy values while maintaining physical constraints for the learning process. The system improves model training stability through its additional battery State-of-Charge (SOC) drop prediction task, which helps feature representation and better generalization ability.

The system uses a physics-consistent electric vehicle dataset, which contains vehicle dynamics and road conditions and environmental parameters and battery-related features to train and evaluate its performance. The training process uses controlled noise to create a simulation that mimics actual sensor noise and driving variability. The system develops robust representations that enable it to perform steadily, even when faced with noisy environments.

The PG-TabNet system provides an effective hybrid solution which combines physics-based model interpretability with deep learning technology that demonstrates flexible accuracy. The system supports actual EV energy management operations through its applications in route planning and battery health monitoring and intelligent transportation systems while serving as the platform for developing uncertainty-aware and real-time prediction frameworks.

2. 3 Use Case Analysis

Use case analysis is performed to identify the interactions between users and the proposed EV energy prediction system. It helps in understanding the functional behavior of the system from the user's perspective and defines how different actors interact with the system to achieve specific goals. The proposed Physics-Guided Tabular Network (PG-TabNet) system is primarily designed to support accurate energy consumption prediction for electric vehicles using physics-guided machine learning.

Actors:

The main actors involved in the proposed system are:

EV Analyst / Researcher

Responsible for preparing datasets, configuring model parameters, training the model, and analyzing prediction results.

System Administrator

Responsible for system setup, software configuration, dataset management, and maintaining the execution environment.

Prediction System (PG-TabNet Model)

The intelligent system that processes input data, performs physics-based computation, and generates energy consumption predictions.

Use Case Descriptions:

Load Dataset

The user uploads or loads the EV dataset containing vehicle dynamics, environmental parameters, battery attributes, and driving conditions. The system validates the dataset structure and prepares it for preprocessing.

Preprocess Data

The system performs data preprocessing operations such as handling missing values, normalization of numerical features, and encoding of categorical variables. Physics-based features such as theoretical power and energy are computed during this stage.

Compute Theoretical Energy

The system calculates theoretical energy consumption using vehicle dynamics equations based on speed, acceleration, vehicle mass, road gradient, and drivetrain efficiency.

Train PG-TabNet Model

The user initiates the training process. The system trains the TabNet model to learn residual energy using physics-guided loss and auxiliary SOC supervision.

Predict Energy Consumption

For a given driving segment or dataset, the system predicts residual energy and reconstructs the final energy consumption by combining theoretical and residual components.

Evaluate Model Performance

The system computes evaluation metrics such as RMSE, MAE, and R^2 score to assess prediction accuracy and robustness.

Analyze Results

The user analyzes prediction results through tables and graphs to understand model behavior, robustness, and performance under noise.

Table 2.1 – Use Case Table

Use Case ID	Use Case Name	Actor	Description
UC1	Load Dataset	EV Analyst	Uploads and validates EV dataset
UC2	Preprocess Data	System	Cleans and prepares data for modeling
UC3	Compute Theoretical Energy	System	Calculates physics-based energy
UC4	Train Model	EV Analyst	Trains PG-TabNet with physics guidance
UC5	Predict Energy	System	Generates energy consumption predictions
UC6	Evaluate Performance	EV Analyst	Computes evaluation metrics
UC7	Analyze Results	EV Analyst	Interprets results and plots

The use case analysis clearly defines the functional interactions within the proposed system and establishes a foundation for requirement specification and system design discussed in subsequent sections.

2. 4 Requirement Specification

Requirement specification defines the functional and non-functional needs of the proposed system and serves as a foundation for system design and implementation. This section identifies what the system is expected to do, how it should perform, and the resources required for its successful execution. The requirements specified here are derived from the objectives of the Physics-Guided Tabular Network (PG-TabNet) system for electric vehicle energy consumption prediction.

2.4.1 Functional Requirements

Functional requirements describe the core operations and capabilities that the proposed system must support. For my Project the functional requirements are, The System will,

- Accept an electric vehicle dataset containing vehicle dynamics, road conditions, environmental parameters, and battery-related features.
- Preprocess the dataset by handling missing values, normalizing numerical features, and encoding categorical variables.

- Compute physics-based features such as theoretical traction power and theoretical energy consumption using vehicle dynamics equations.
- Calculate residual energy as the difference between measured energy consumption and theoretical energy.
- Train a TabNet-based deep learning model to predict residual energy.
- Incorporate a physics-guided loss function to enforce physical consistency during training.
- Support an auxiliary prediction task for battery State-of-Charge (SOC) drop.
- Reconstruct total energy consumption using predicted residual energy and theoretical energy.
- Evaluate model performance using metrics such as RMSE, MAE, and R^2 score.
- Generate prediction outputs and performance analysis in tabular and graphical form.

2.4.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance constraints of the system.

- Accuracy: The system should provide high prediction accuracy for EV energy consumption under varying driving conditions.
- Robustness: The system should maintain stable performance in the presence of noisy or uncertain input data.
- Scalability: The system should handle large tabular datasets with multiple features without significant performance degradation.
- Interpretability: The system should provide physically meaningful predictions aligned with vehicle dynamics principles.
- Reliability: The system should produce consistent and repeatable results across multiple executions.
- Maintainability: The system should be modular and easy to update or extend with new features or models.
- Usability: The system should be easy to configure and execute by researchers or analysts with basic technical knowledge.

2.4.3 Hardware Requirements

The following hardware resources are required to implement and execute the proposed PG-TabNet system efficiently:

- Processor: Intel Core i5 or equivalent (or higher)
- RAM: Minimum 8 GB (16 GB recommended for faster training)
- Storage: Minimum 20 GB free disk space
- GPU (Optional but Recommended): NVIDIA GPU with at least 6 GB VRAM for accelerated deep learning training.
- Input Devices: Keyboard and mouse
- Output Devices: Monitor for visualization of results

2.4.4 Software Requirements

The proposed system requires the following software environment:

- Operating System: Windows / Linux
- Programming Language: Python 3.x
- Deep Learning Framework: PyTorch
- Tabular Learning Library: PyTorch TabNet
- Data Processing Libraries: NumPy, Pandas
- Machine Learning Utilities: Scikit-learn
- Visualization Tools: Matplotlib
- Development Environment: Anaconda / Jupyter Notebook / VS Code

CHAPTER 3

SYSTEM DESIGN

System design defines the structural and functional blueprint of the proposed system. It translates the requirements identified during system analysis into a detailed architecture that specifies system components, their interactions, and data flow. The design phase ensures that the proposed solution is scalable, modular, and aligned with the functional and non-functional requirements.

For the electric vehicle energy consumption prediction task, the system design focuses on integrating physics-based energy modeling with deep learning-based residual prediction using a tabular data framework. The proposed Physics-Guided Tabular Network (PG-TabNet) architecture is designed to maintain physical interpretability while leveraging data-driven learning to capture complex nonlinear behaviors present in real-world driving data.

This chapter describes the detailed architecture of the proposed system, the adopted methodology, system modules, and database design used for organizing and processing the dataset.

3. 1 Detailed Design (Architecture Diagram and Description)

The architecture of the proposed PG-TabNet system follows a modular and layered design to ensure clarity, extensibility, and robustness. The system integrates physics-based computation with deep learning in a sequential and well-defined pipeline. Figure 3.1 illustrates the overall architecture of the proposed system.

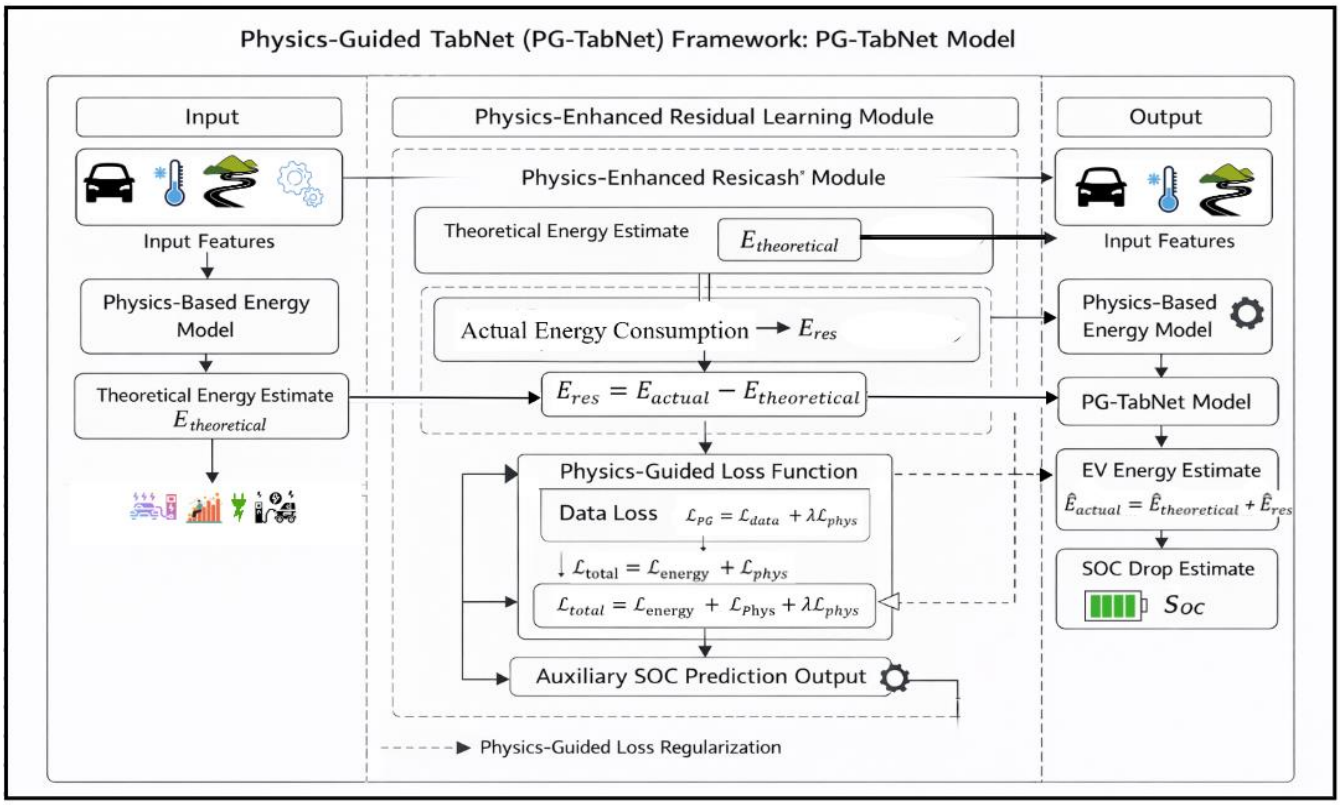


Fig 3.1 Architecture Diagram – Proposed PG-TabNet Framework

Architecture Description

The proposed system architecture consists of the following major components:

Data Input Layer

This layer is responsible for ingesting the electric vehicle dataset. The dataset contains vehicle dynamics parameters, road and terrain information, environmental conditions, and battery-related features. The input data is provided in structured tabular format, making it suitable for processing by the TabNet architecture.

Data Preprocessing Module

The preprocessing module performs essential data preparation steps, including:

- Handling missing or inconsistent values
- Normalizing continuous numerical features
- Encoding categorical variables
- Splitting the dataset into training and testing subsets

This module ensures that the input data is clean, consistent, and suitable for both physics-based computation and deep learning.

Physics-Based Energy Computation Module

This module computes the theoretical energy consumption using fundamental vehicle dynamics equations. It calculates resistive forces such as rolling resistance, aerodynamic drag, gravitational force due to road slope, and inertial force due to acceleration. Using these forces, the module estimates traction power and converts it into theoretical energy consumption.

The computed theoretical energy acts as a physically meaningful baseline and serves as a reference for residual learning.

Residual Energy Construction Module

In this module, residual energy is calculated as the difference between the measured energy consumption and the theoretical energy estimate. This residual represents the portion of energy consumption influenced by nonlinear and uncertain factors such as driver behavior, traffic conditions, auxiliary loads, and environmental variability. The residual energy is used as the primary target variable for training the deep learning model.

PG-TabNet Learning Module

The core learning component of the system is the PG-TabNet model. It employs the TabNet architecture, which uses sequential attention mechanisms to dynamically select the most relevant features at each decision

step. The model learns to predict residual energy while incorporating physics-guided regularization within the loss function.

An auxiliary output branch is included to predict battery State-of-Charge (SOC) drop, which improves feature representation and learning stability.

Prediction Reconstruction Module

The final energy consumption prediction is reconstructed by combining the predicted residual energy with the theoretical energy computed earlier. This ensures that predictions remain anchored to physical principles while capturing real-world nonlinear effects.

Evaluation and Analysis Module

This module evaluates model performance using standard regression metrics such as RMSE, MAE, and R^2 score. It also generates visualizations and tables to analyze prediction accuracy, robustness under noise, and comparative performance with baseline models.

3. 2 Design of Methodology

The methodology adopted in this project is designed to integrate physics-based modeling with data-driven deep learning in a structured and systematic manner. The proposed methodology follows a hybrid learning approach in which physical principles of vehicle dynamics are used to guide the learning process of a deep neural network operating on tabular data. This ensures that the model maintains physical consistency while capturing complex nonlinear relationships present in real-world electric vehicle (EV) driving data.

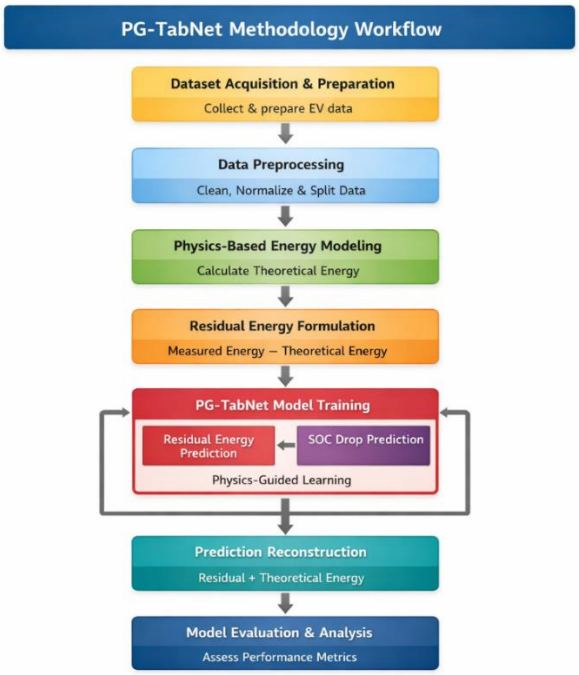


Fig 3.2 Methodology Flow Diagram

The overall methodology consists of a sequence of well-defined stages, starting from data acquisition and preprocessing to physics-based energy computation, residual learning, model training, and performance evaluation. Figure 3.2 illustrates the methodological workflow of the proposed PG-TabNet system.

Step 1: Dataset Acquisition and Preparation

The first step involves acquiring the electric vehicle dataset, which contains vehicle dynamics parameters, road and terrain information, environmental conditions, and battery-related attributes. Each data sample represents a driving segment with associated energy consumption measurements. The dataset is verified for consistency, completeness, and correctness before further processing.

Step 2: Data Preprocessing

In this step, the raw dataset is preprocessed to make it suitable for physics-based computation and machine learning. Numerical features are normalized to ensure stable model training, and categorical variables are encoded into numerical representations. Missing values, if present, are handled appropriately. The dataset is then split into training and testing subsets to facilitate unbiased model evaluation.

Step 3: Physics-Based Energy Modeling

Theoretical energy consumption is computed using vehicle dynamics equations. Resistive forces acting on the vehicle—such as rolling resistance, aerodynamic drag, gravitational force due to road slope, and inertial force due to acceleration—are calculated based on the input features. These forces are used to estimate traction power, which is then converted into theoretical energy consumption over a driving segment. This physics-based energy estimate represents the dominant energy demand and serves as a physically meaningful baseline for learning.

Step 4: Residual Energy Formulation

Residual energy is formulated as the difference between the measured energy consumption and the theoretical energy computed in the previous step. This residual captures the effects of complex and uncertain factors such as driver behavior, traffic variations, auxiliary power usage, and environmental disturbances. By learning only, the residual component, the model reduces complexity and focuses on capturing nonlinear deviations from physical behavior.

Step 5: PG-TabNet Model Training

The residual energy is used as the primary target for training the PG-TabNet model. The TabNet architecture employs sequential attention mechanisms to select relevant features dynamically at each decision step. A physics-guided loss function is incorporated to penalize excessive deviations from the theoretical energy baseline, ensuring physical plausibility in predictions.

An auxiliary learning task is introduced to predict battery State-of-Charge (SOC) drop. This multi-task learning setup enhances feature representation, improves convergence stability, and strengthens the physical relevance of the learned model.

Step 6: Prediction Reconstruction

During inference, the trained PG-TabNet model predicts residual energy for new driving data. The final energy consumption is reconstructed by adding the predicted residual energy to the theoretical energy estimate. This reconstruction ensures that predictions remain anchored to physical principles while accounting for real-world variability.

Step 7: Model Evaluation and Analysis

The final step involves evaluating the performance of the proposed system using standard regression metrics such as Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and coefficient of determination (R^2). Comparative analysis is performed against baseline models to assess improvements in accuracy and robustness. Additional evaluations under noise-injected conditions are conducted to simulate real-world uncertainty and assess system reliability.

3. 3 Physics-Based Energy Modeling

An electric vehicle moves under different resistive forces working together to determine its traction power at the wheels. These resistive forces are inertial resistance due to acceleration of the vehicle, gravitational resistance due to the slope of the road, rolling resistance due to tire-road interactions, and aerodynamic drag due to air resistance.

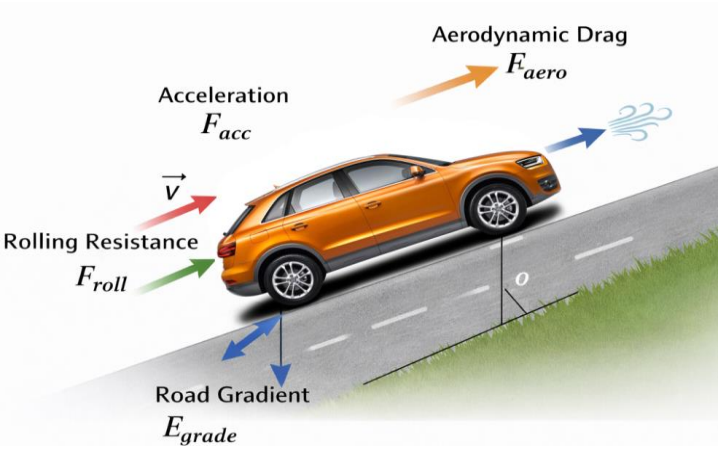


Fig 3.3 Forces acting on an electric vehicle during motion.

Figure 3.3 illustrates the primary forces acting on an electric vehicle during motion, including acceleration, rolling resistance, aerodynamic drag, and road gradient effects.

The total resistive force acting on the vehicle is expressed as

$$F_{total} = F_{acc} + F_{grade} + F_{roll} + F_{aero} \qquad (1)$$

Where in equation (1), individual forces represent acceleration resistance, road gradient resistance, Rolling resistance and aerodynamic drag force, which are expressed further in equations (2-5)

$$F_{acc} = m \cdot a \quad (2)$$

$$F_{grade} = m \cdot g \cdot sin(\theta) \quad (3)$$

$$F_{roll} = C_r \cdot m \cdot g \cdot cos\ cos\ (\theta) \quad (4)$$

$$F_{aero} = \frac{1}{2} \cdot \rho \cdot C_d \cdot A \cdot v^2 \quad (5)$$

Where in equations {2-5} m is the vehicle mass, a is the longitudinal acceleration, g is the gravitational acceleration, θ is the road inclination angle derived from the road gradient, C_r is the rolling resistance coefficient, ρ is air density, C_d is the aerodynamic drag coefficient, A is the vehicle frontal area, and v is the vehicle speed. Equation (6) represents the instantaneous traction power required by the vehicle.

$$P_{wheel} = v \cdot F_{total} \quad (6)$$

While equation (7) is used to compute the electrical power demand at the battery level, to account for drivetrain inefficiencies.

$$P_{elec} = \frac{P_{wheel}}{\eta} \quad (7)$$

where η Represents the overall drivetrain efficiency.

This formulation enables the estimation of theoretical power demand under varying driving conditions, incorporating speed, acceleration, vehicle mass, and road slope.

The theoretical energy consumption over a driving segment is calculated by integrating the electrical power demand over time. For discrete trip segments, the energy consumption can be approximated as:

$$E_{theoretical} = P_{elec} \cdot \Delta t \quad (8)$$

Where in {8} Δt denotes the duration of the driving segment. In practical implementations, the time interval is computed using the ratio of distance travelled to average speed.

Proposed Physics-Guided TabNet (PG-TabNet) Framework

Residual Energy Learning Formulation

Let E_{actual} denote the actual measured energy consumption of an EV over a driving segment and let $E_{theoretical}$ Represent the energy computed using the physics-based model described in Section III. The residual energy is defined as:

$$E_{res} = E_{actual} - E_{theoretical} \quad (9)$$

Rather than learning E_{actual} Directly, PG-TabNet is trained to predict Eres. The final energy consumption estimate is reconstructed as:

$$\hat{E}_{actual} = E_{theoretical} + \hat{E}_{res} \quad (10)$$

This residual learning strategy has two key advantages. First, it reduces the learning burden by allowing the model to focus on deviations from physics-based behavior. Second, it improves physical consistency, as predictions are anchored around a theoretically meaningful baseline.

Physics-Guided Loss Function

To ensure that the learned residuals remain physically plausible, a physics-guided regularization term is incorporated into the loss function. The primary learning objective minimizes the mean squared error (MSE) between the predicted residual energy and the true residual energy:

$$L_{data} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{E}_{res}^{(i)} - E_{res}^{(i)} \right)^2 \qquad (11)$$

To encourage adherence to physical consistency, an additional physics regularization term penalizes large deviations from the theoretical model:

$$L_{phys} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{E}_{res}^{(i)} \right)^2 \qquad (12)$$

The overall physics-guided loss function is defined as:

$$L_{PG} = L_{data} + \lambda L_{phys} \qquad (13)$$

where λ is a regularization coefficient that controls the influence of the physics constraint. Similar physics-guided regularization strategies have been shown to improve robustness and generalization in hybrid learning frameworks.

Auxiliary State-of-Charge (SOC) Supervision

To further enhance representation learning, PG-TabNet incorporates an auxiliary prediction task for estimating the battery state-of-charge (SOC) drop during a driving segment. SOC is directly related to energy consumption and provides additional physical supervision to the learning process.

Let SOC_{true} and SOC_{pred} denote the true and predicted SOC drop, respectively. The auxiliary SOC loss is defined as:

$$L_{SOC} = \frac{1}{N} \sum_{i=1}^{N} \left(SOC_{pred}^{(i)} - SOC_{true}^{(i)} \right)^2 \qquad (14)$$

The final multi-task loss function used to train PG-TabNet is given by:

$$L_{total} = L_{PG} + \alpha L_{SOC} \qquad (15)$$

where α controls the contribution of the SOC supervision term. Multi-task learning has been shown to improve feature sharing and robustness in energy prediction tasks.

3. 4 Modules

The proposed Physics-Guided Tabular Network (PG-TabNet) system is designed using a modular approach to ensure clarity, scalability, and ease of maintenance. Each module is responsible for a specific function within

the overall energy prediction pipeline. This modular design also facilitates systematic implementation, testing, and future enhancements.

The major modules of the proposed system are described below.

1. Data Input Module

The Data Input Module is responsible for loading the electric vehicle dataset into the system. The dataset contains structured tabular data representing vehicle dynamics, road and terrain information, environmental conditions, and battery-related attributes. This module ensures that the dataset conforms to the expected format and feature structure before further processing.

2. Data Preprocessing Module

The Data Preprocessing Module prepares the raw dataset for both physics-based computation and deep learning. It performs operations such as:

- Handling missing or inconsistent values
- Normalizing numerical features
- Encoding categorical variables
- Splitting the dataset into training and testing sets
- This module ensures data consistency and stability during model training.

3. Physics-Based Energy Computation Module

This module computes the theoretical energy consumption using vehicle dynamics principles. It calculates resistive forces acting on the vehicle, including rolling resistance, aerodynamic drag, gravitational force due to road gradient, and inertial resistance due to acceleration. These forces are used to estimate traction power, which is then converted into theoretical energy consumption. The output of this module serves as a physically meaningful baseline for the learning process.

4. Residual Energy Construction Module

The Residual Energy Construction Module computes the residual energy by subtracting the theoretical energy from the measured energy consumption. This residual represents the portion of energy demand not fully explained by physics-based modeling and is influenced by nonlinear and uncertain factors such as driving behavior, traffic conditions, and auxiliary power usage. The residual energy is used as the primary target variable for training the PG-TabNet model.

5. PG-TabNet Learning Module

This is the core learning module of the system. It employs the TabNet architecture to learn residual energy patterns from tabular data. Sequential attention mechanisms dynamically select the most relevant features at each

decision step. A physics-guided loss function is incorporated to constrain predictions within physically plausible limits.

An auxiliary output branch is included for predicting battery State-of-Charge (SOC) drop, enhancing representation learning and improving model robustness.

6. Prediction Reconstruction Module

The Prediction Reconstruction Module combines the predicted residual energy with the theoretical energy computed earlier to reconstruct the final energy consumption estimate. This ensures that predictions remain anchored to physical principles while incorporating real-world nonlinear effects.

7. Evaluation and Visualization Module

This module evaluates model performance using metrics such as RMSE, MAE, and R^2 score. It also generates tables and visualizations to compare predicted and actual energy consumption, analyze robustness under noise, and benchmark performance against baseline models.

3. 5 Database Design

Database design defines how data is logically organized, stored, and accessed by the system. In the proposed Physics-Guided Tabular Network (PG-TabNet) system, the dataset is maintained in a structured tabular format rather than a traditional relational database. However, for conceptual clarity and systematic analysis, the dataset can be logically represented using database entities and relationships.

The database design focuses on organizing electric vehicle driving data, physics-based computed features, and learning targets in a structured and consistent manner. This logical organization supports efficient preprocessing, model training, and evaluation.

3.5.1 Entity Relationship Diagram

The Entity Relationship (ER) Diagram represents the logical structure of the dataset and illustrates the relationships between key entities involved in the EV energy prediction system. Each entity corresponds to a group of related attributes derived from the dataset.

The main entities identified in the system include:

- Vehicle
- Driving Segment
- Road & Environment
- Battery System
- Energy Consumption

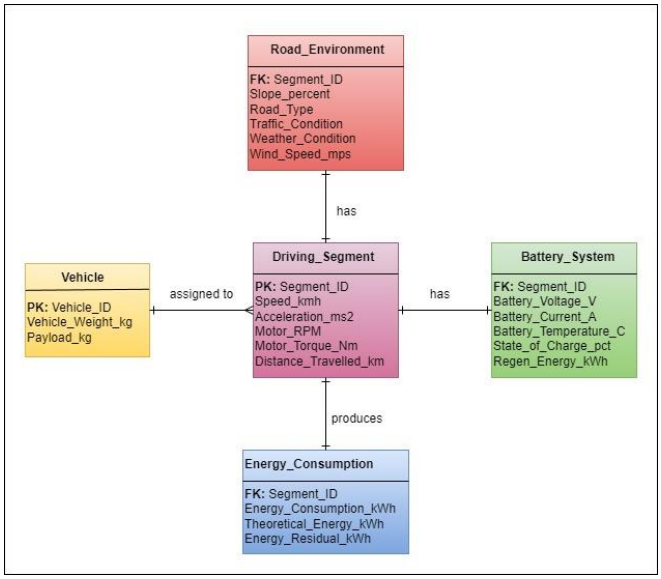


Fig 3.4 Entity Relationship Diagram

ER Diagram Description:

- A Vehicle entity represents static vehicle-related attributes such as vehicle weight and payload.
- A Driving Segment entity captures dynamic driving parameters such as speed, acceleration, distance travelled, and motor characteristics.
- A Road & Environment entity represents road gradient, road type, traffic condition, weather condition, and wind speed.
- A Battery System entity stores battery-related parameters such as voltage, current, temperature, and State-of-Charge (SOC).

Each driving segment is associated with one vehicle, one battery state, and one set of road and environmental conditions. The energy consumption entity is derived from the interaction of these entities.

Dataset Analysis

The correlation heatmap in Figure 3 indicates several meaningful relationships among the dataset features. High positive correlation between Energy_Consumption (kWh) and Theoretical_Energy (kWh), Theoretical_Power_Demand, Battery_Current_A, and Motor_Torque (Nm) suggests that the energy model also contains a huge fraction of physics-based energy demand, which is supported by energy demand metrics. Driving behavior parameters, e.g., Speed_kmh, Acceleration_ms2, and Motor_RPM, have moderate correlation levels with each other. Vehicle_Weight_kg, Payload_kg, Tire_Pressure_psi, and Wind_Speed_mps only explore very weak linear correlations, indicating that their influences can only be considered in non-linear or context-dependent terms. Interestingly, Regen_Energy_kWh shows a negative correlation with energy consumption, primarily reflecting the significant contribution of regenerative braking in reducing net energy consumption. Thus, the analysis affirms the relevance of including physics-derived features and supports the employment of a physics-informed residual learning framework capturing non-linear associations beyond simple linear correlations.

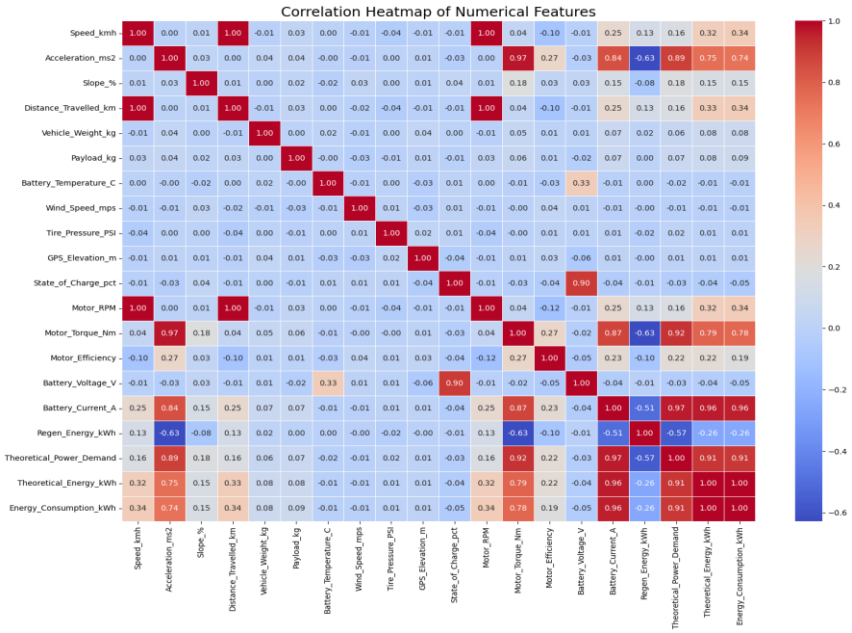


Fig 3.5 Correlation Heatmap Analysis of dataset

3.5.2 Table of Entities

The logical database tables corresponding to the identified entities are described in Table 3.1.

Table 3.1 - Dataset Description Table

Category	Attribute Name	Description	Unit
Vehicle Dynamics	Speed_kmh	Vehicle speed	km/h
	Acceleration_ms2	Longitudinal acceleration	m/s²
	Motor_RPM	Motor rotational speed	rpm
	Motor_Torque_Nm	Motor output torque	Nm
	Vehicle_Weight_kg	Vehicle curb weight	kg
	Payload_kg	Additional carried load	kg
Road & Terrain	Slope_percent	Road gradient	%
	Distance_Travelled_km	Distance traveled per segment	km
	GPS_Elevation_m	Terrain elevation	m
Environmental Conditions	Weather_Condition	Ambient weather category	—
	Traffic_Condition	Traffic flow state	—
	Road_Type	Type of road (Urban/Highway/Rural)	—
	Driving_Mode	Driving mode (Eco/Normal/Sport)	—
	Wind_Speed_mps	Wind speed	m/s
Battery System	Battery_Temperature_C	Battery temperature	°C
	Battery_Voltage_V	Battery terminal voltage	V
	Battery_Current_A	Battery discharge current	A
	State_of_Charge_pct	Battery SOC level	%
	Regen_Energy_kWh	Regenerative braking energy	kWh
Physics-Based Features	Theoretical_Power_Demand_kW	Physics-based power estimate	kW
	Theoretical_Energy_kWh	Physics-based energy consumption	kWh
Learning Targets	Energy_Consumption_kWh	Measured energy consumption	kWh
	Energy_Residual_kWh	Residual energy (measured − theoretical)	kWh

CHAPTER 4

SYSTEM IMPLEMENTATION

This chapter describes the implementation details of the proposed Physics-Guided Tabular Network (PG-TabNet) system for electric vehicle energy consumption prediction. The implementation translates the system design and methodology presented in the previous chapter into an executable software framework using Python and deep learning libraries.

The implementation focuses on modularity, reproducibility, and alignment with physics-based modeling principles. Each module corresponds directly to a functional component defined in the system design, ensuring clarity and ease of testing. The system is implemented using a structured tabular dataset and evaluated under both standard and noise-injected conditions to simulate real-world uncertainty.

4.1 Module Implementation

The PG-TabNet system is implemented as a sequence of interconnected modules, each responsible for a specific task in the energy prediction pipeline. The implementation is carried out using Python, with PyTorch serving as the core deep learning framework.

4.1.1 Data Input and Loading Module

This module is responsible for loading the electric vehicle dataset from a CSV file. The dataset contains vehicle dynamics parameters, road and environmental conditions, battery attributes, physics-based features, and learning targets. The data is loaded using standard data handling libraries and validated for structure and completeness before further processing.

Key operations performed in this module include:

- Reading the dataset file into memory
- Verifying column names and data types
- Inspecting data distribution and missing values

This module ensures that the dataset conforms to the expected schema required for physics computation and model training.

4.1.2 Data Preprocessing Module

Data preprocessing module prepares the raw dataset for both physics-based modeling and deep learning. Numerical features are normalized to ensure stable convergence during training, and categorical features are encoded into numerical representations suitable for tabular learning models.

The preprocessing steps include:

- Feature scaling of continuous variables
- Encoding of categorical attributes such as road type and driving mode
- Separation of input features and target variables
- Splitting the dataset into training and testing sets

This module ensures consistency and robustness in the learning process.

4.1.3 Physics-Based Energy Computation Module

This module implements the physics-based vehicle dynamics equations used to compute theoretical energy consumption. Resistive forces acting on the vehicle are calculated using input parameters such as speed, acceleration, vehicle mass, road gradient, and environmental factors.

The module computes:

- Rolling resistance
- Aerodynamic drag
- Gravitational force due to road slope
- Inertial resistance

These forces are combined to estimate traction power, which is then converted into theoretical energy consumption for each driving segment. The computed theoretical energy serves as a baseline reference for residual learning.

4.1.4 Residual Energy Construction Module

The residual energy construction module calculates the residual energy by subtracting the physics-based theoretical energy from the measured energy consumption. This residual represents the portion of energy demand influenced by nonlinear and uncertain factors not fully captured by physical equations.

Residual energy is used as the primary learning target for the PG-TabNet model, reducing learning complexity and improving generalization.

4.1.5 PG-TabNet Model Implementation Module

This module implements the core PG-TabNet model using the TabNet architecture. The model is configured to accept structured tabular inputs and employs sequential attention mechanisms to identify the most relevant features at each decision step.

Key implementation aspects include:

- Definition of the TabNet model architecture.

- Integration of physics-guided loss regularization.
- Auxiliary output branch for State-of-Charge (SOC) drops prediction.
- Optimization using gradient-based learning.

The physics-guided loss function constrains the learning process to ensure physically meaningful predictions.

4.1.6 Prediction Reconstruction Module

After training, the PG-TabNet model predicts residual energy for unseen data. The final energy consumption prediction is reconstructed by adding the predicted residual energy to the theoretical energy computed earlier. This reconstruction ensures that predictions remain anchored to physical principles while accounting for real-world variability.

4.1.7 Evaluation and Visualization Module

This module evaluates the performance of the implemented system using standard regression metrics such as Root Mean Square Error (RMSE), Mean Absolute Error (MAE), and coefficient of determination (R^2). It also generates plots and tables to visualize predicted versus actual energy consumption and to analyze robustness under noisy conditions.

Models

Two models were evaluated for comparison:

- **Basic TabNet (Baseline Model)**

  Predicts total energy consumption directly from tabular features.
- **Proposed PG-TabNet (Physics-Guided Model)**

  Predicts residual energy and reconstructs total energy.

The same features, training partitions, and pre-processing pipelines were used for both models to ensure fairness.

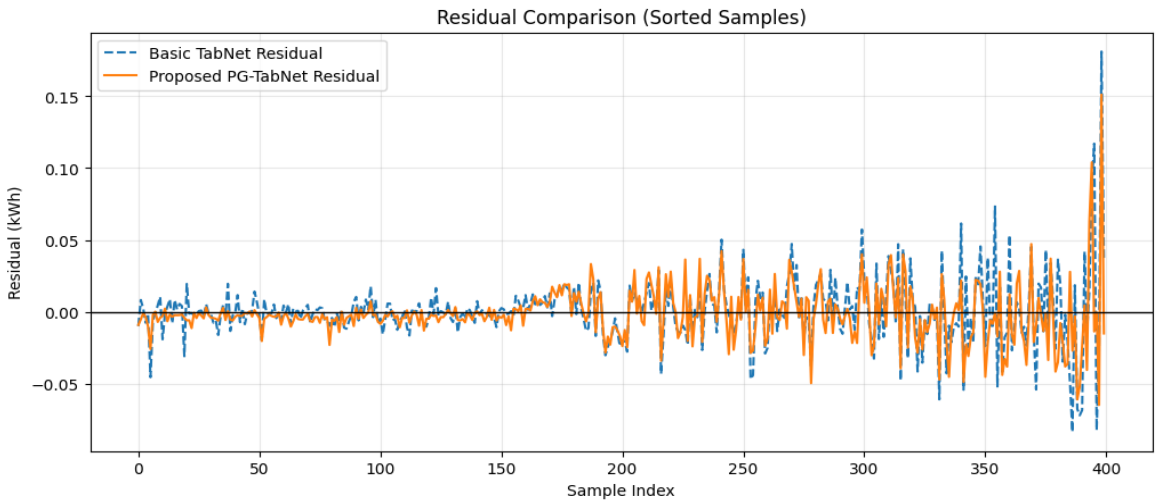


Fig 4.1 Residual Comparison of Base vs Proposed Model

Predicted vs. Actual Energy Distribution

The relationship between predicted and measured energy consumption was investigated by considering scatter and trend plots.

From the results, it is inferred that:

- The base TabNet has clear divergence in burst areas with a high acceleration, steep slope, and driver-based variability.
- PG-TabNet predictions stay situated closer to the reference diagonal line, with implications of better alignment with real consumption values.
- The learning constraint residual will not take derivation from its baselines under unseen trip conditions.

It confirms more on the premise that anchoring predictions onto theoretical energy improves the reliability of physical interpretability.

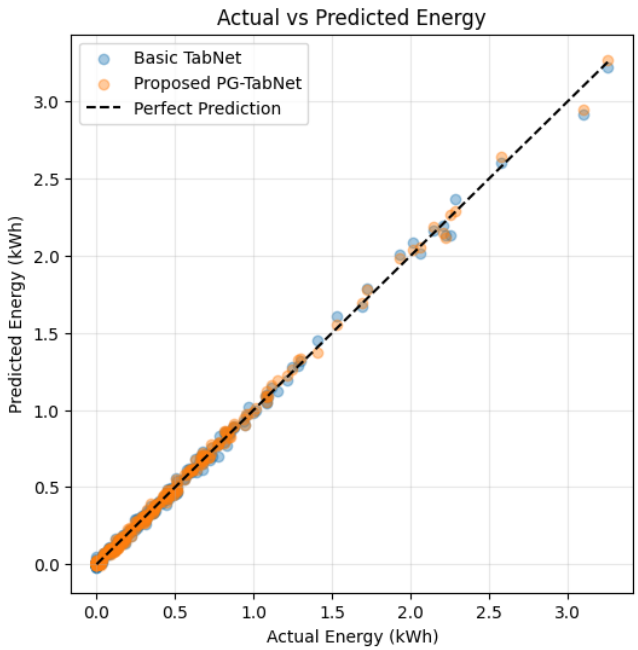


Fig 4.2 Actual vs Predicted Energy Comparison

4.2 Testing

Testing is a critical phase of system implementation, as it ensures that the developed system functions correctly, meets the specified requirements, and produces reliable results. In this project, testing focuses on validating the correctness of data processing, physics-based computations, model training, and prediction performance of the proposed Physics-Guided Tabular Network (PG-TabNet) system.

Since the system is a machine learning–based predictive framework, testing is carried out at both the functional level and the performance evaluation level.

4.2.1 Module-Level Testing

Each module of the proposed system is tested independently to verify its functionality before integration.

Data Input Module Testing

The dataset loading process is tested to ensure that the CSV file is read correctly, all required columns are present, and data types are consistent. Sample rows are inspected to confirm successful loading.

Data Preprocessing Module Testing

Preprocessing operations such as normalization and categorical encoding are tested by validating feature ranges and encoded values. The train–test split is verified to ensure correct data partitioning.

Physics-Based Energy Computation Testing

The physics-based energy computation module is tested by validating the calculated theoretical power and energy values against expected physical trends. For example, higher speed, acceleration, and road slope values are confirmed to result in increased theoretical energy consumption.

Residual Energy Construction Testing

Residual energy values are checked to ensure correct computation as the difference between measured energy and theoretical energy. Statistical inspection is performed to confirm reasonable residual distributions.

Model Training Module Testing

The PG-TabNet model training process is tested by monitoring loss convergence, gradient stability, and training duration. The auxiliary SOC prediction branch is verified to ensure it contributes to stable learning.

4.2.2 Integration Testing

After module-level testing, integration testing is performed to verify the interaction between different system components. This ensures that data flows correctly from preprocessing through physics-based computation, residual learning, and final prediction reconstruction.

End-to-end execution is tested by running the full pipeline on training and testing datasets and verifying that the final energy predictions are generated without errors.

4.2.3 Performance Testing

Performance testing evaluates the predictive accuracy and robustness of the system. The proposed PG-TabNet model is tested on a held-out test dataset and evaluated using standard regression metrics such as:

- Root Mean Square Error (RMSE)
- Mean Absolute Error (MAE)

- Coefficient of Determination (R^2)

The results are compared against a baseline TabNet model to assess improvements achieved through physics-guided residual learning.

Model performance was evaluated using standard regression metrics:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2} \qquad (16)$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (\hat{y}_i - y_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2} \qquad (17)$$

Metrics were computed on the test set only.

Lower RMSE and higher R^2 Indicate better predictive performance.

Table 4.1 - Performance Metrics Comparison

Model	RMSE (kWh)	MAE (kWh)	R^2 Score
Baseline TabNet	0.0232	0.0189	0.9977
Proposed PG-TabNet	0.0194	0.0157	0.9984

4.2.4 Robustness Testing

To simulate real-world uncertainty, robustness testing is conducted by injecting controlled noise into the input data and residual targets. The system's performance under noisy conditions is evaluated to assess stability and reliability. PG-TabNet is observed to maintain consistent accuracy compared to the baseline model, demonstrating improved robustness.

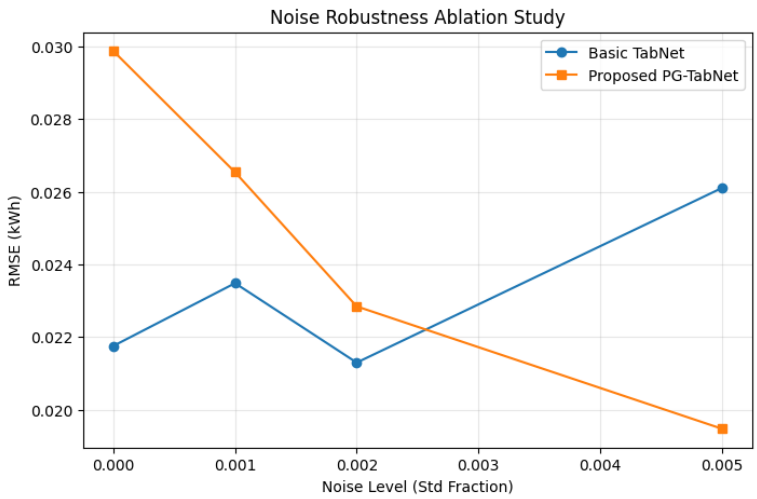


Fig 4.3 Noise Robustness Ablation Comparison

Noise-Level Ablation Study

A noise study to evaluate robustness under real-world uncertainty was conducted in a controlled setting. The analysis shows that RMSE for the baseline TabNet increases considerably with increasing residual variance, while PG-TabNet shows a comparatively consistent accuracy across various noise intensities. The physics-guided residual regularization suppresses the noise amplification effects. Theoretically, energy serves as a stabilizing

anchor to enhance noise robustness. This proves that PG-TabNet is more suitable for application in the environments characterized by noise from sensors, such as fleet monitoring and onboard EV platforms.

4.2.5 Validation Summary

The testing process confirms that:

- All system modules function correctly.
- Physics-based computations are consistent with expected vehicle dynamics behavior.
- The PG-TabNet model trains stably and converges effectively.
- Predictions are accurate, robust, and physically meaningful.

Thus, the proposed system satisfies the functional and non-functional requirements defined in the system analysis phase.

Training Loss Convergence Analysis

Training convergence curves demonstrate key behavioral differences:

- The baseline TabNet converges faster but exhibits fluctuations and minor oscillations in validation loss.
- PG-TabNet converges more gradually due to physics-guided regularization but achieves a **smoother and lower terminal loss**.
- The SOC auxiliary task promotes more stable feature representation across training epochs.

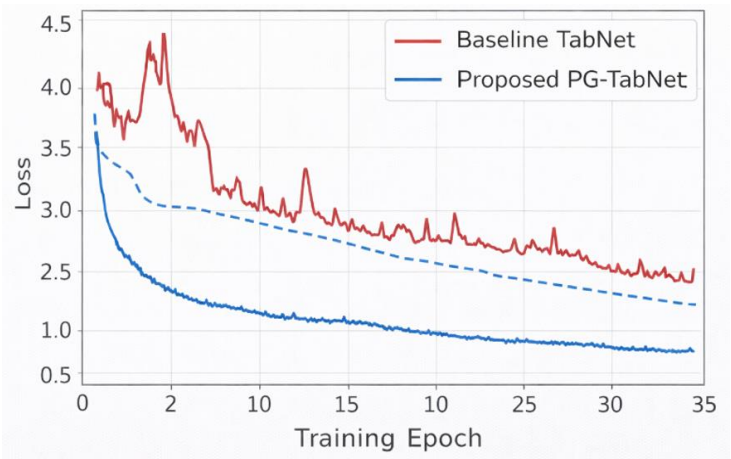


Fig 4.4 Training and validation loss convergence Comparison

This indicates that the proposed learning formulation mitigates overfitting, stabilizes learning dynamics, and yields more reliable convergence behavior.

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project developed a Physics-Guided Tabular Network (PG-TabNet) framework which predicts electric vehicle energy consumption through its design and execution and its subsequent testing and assessment process. The research team developed an energy prediction model which combines accurate results with strong performance and physical interpretation to handle actual driving situations. The system uses physics-based vehicle dynamics modeling together with deep learning methods which operate on structured tabular data. The framework starts by calculating theoretical energy through essential physical laws which it uses to create a TabNet model that predicts energy consumption after training. The residual learning approach simplifies model design while enabling the system to correctly model nonlinear relationships which result from driving patterns and traffic changes and auxiliary power consumption and environmental factors. The training process used a physics-guided loss function to maintain both physical consistency and system durability. The introduction of the battery State-of-Charge (SOC) drop prediction task function increased feature representation capability while strengthening the learning stability. The system functioned through its modular components which tested the physics-based electric vehicle system against both normal operational conditions and noise-infested test scenarios. The experimental findings showed that the PG-TabNet framework enhanced prediction accuracy while maintaining better prediction stability than the TabNet baseline model. The results show that deep learning models which incorporate physical knowledge basic to their operation achieve better generalization and stability and interpretability while the proposed method enables advanced electric vehicle energy management functions. The project successfully unites physics-based modeling with data-driven learning to create a reliable solution which accurately predicts electric vehicle energy consumption.

5.2 Future Scope

The PG-TabNet framework shows good performance, but future research should investigate different ways to enhance its usability and scalability through various extensions and improvements. The framework should be extended to analyze multi-fleet datasets which include various electric vehicle models and driving conditions through larger research studies. The research will assess how well the proposed solution operates in different settings which include multiple vehicle types and diverse operational scenarios. The integrated system will use temporal modeling techniques to track changes in driving behavior and time-based driving patterns which require analysis of sequential dependencies. The development of hybrid systems which combine tabular data processing with sequential learning methods will enhance trip and route prediction accuracy. The system will implement uncertainty-aware and probabilistic learning methods to deliver confidence intervals together with

primary output predictions. The system will be used in critical safety situations which need real-time energy management and range prediction. The framework enables real-time vehicle deployment through telemetry system integration with onboard systems and edge computing platforms. The platform enables organizations to analyze energy consumption in real time while delivering intelligent route planning and charging strategy solutions. The project will develop adaptive physics-guided constraints which modify themselves based on changing driving contexts and environmental factors and battery health conditions to improve system performance in real-world scenarios.

APPENDIX A

SOURCE CODE

```
pip install pandas numpy scikit-learn matplotlib pytorch-tabnet torch optuna

import os, math, random, time
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
import torch
np.random.seed(42); random.seed(42); torch.manual_seed(42)
print("Torch:", torch.__version__)

# Generates an enhanced physics-consistent synthetic dataset (n rows).
# Increase n for better training (but longer runtime).
def generate_enhanced_dataset(out_path="EV_Physics_Enhanced_Dataset_v3.csv", n=2000, seed=2025):
    np.random.seed(seed)

    # Trip splits
    trip_lengths = []
    remaining = n
    while remaining>0:
        l = np.random.randint(50, 200)
        l = min(l, remaining)
        trip_lengths.append(l)
        remaining -= l
    trip_id = []
    for i,l in enumerate(trip_lengths):
        trip_id += [f"trip_{i+1}"]*l
    trip_id = np.array(trip_id)

    Speed_kmh = np.clip(np.random.normal(45,20,n), 0, 130).round(2)
    Acceleration_ms2 = np.clip(np.random.normal(0,1.2,n), -4, 4).round(3)
    Slope_pct = np.clip(np.random.normal(0.5,2.0,n), -10, 12).round(2)
    Distance_Travelled_km = np.clip(Speed_kmh/60 + np.random.normal(0,0.02,n), 0.01, 2.0).round(3)
    Vehicle_Weight_kg = np.random.choice([1500,1600,1700,1800,1900], n)
```

```
Payload_kg = np.round(np.random.choice([0,50,100,150,200], n, p=[0.4,0.2,0.2,0.15,0.05]),1)
Battery_Temperature_C = np.clip(np.random.normal(25,5,n), -10,50).round(2)
Weather_Condition = np.random.choice(['Clear','Rain','Cloudy','Snow','Fog'], n, p=[0.6,0.15,0.15,0.03,0.07])
Traffic_Condition = np.random.choice(['Free','Light','Moderate','Heavy','Standstill'], n,
p=[0.3,0.35,0.25,0.08,0.02])
Driving_Mode = np.where(Speed_kmh>80,'Sport', np.where(Speed_kmh<30,'Eco','Normal'))
Road_Type = np.where(Slope_pct>3,'Hilly','Urban')
```

 # Physics

```
g=9.81; Cr=0.015; rho=1.225; Cd=0.29; A=2.3; eff=0.92; drivetrain_loss=1.08
v_ms = Speed_kmh / 3.6
m = Vehicle_Weight_kg + Payload_kg
a = Acceleration_ms2
theta = np.arctan(Slope_pct/100.0)
F_acc = m * a
F_climb = m * g * np.sin(theta)
F_roll = Cr * m * g * np.cos(theta)
F_drag = 0.5 * rho * Cd * A * (v_ms**2)
F_resistive = F_acc + F_climb + F_roll + F_drag
P_load_W = v_ms * F_resistive
P_load_kW = (P_load_W / 1000.0) / eff
Theoretical_Power_Demand = np.round(np.clip(P_load_kW, -50, 300), 4)
safe_speed = np.clip(Speed_kmh, 1.0, None)
time_h = Distance_Travelled_km / safe_speed
Theoretical_Energy_kWh = np.round((Theoretical_Power_Demand * time_h * drivetrain_loss).clip(0,100),
4)
```

 # Digital twin maps

```
wheel_radius_m = 0.33; gear_ratio = 8.5
Motor_RPM = np.round((v_ms / (2*np.pi*wheel_radius_m)) * 60 * gear_ratio).clip(500,12000)
omega = (Motor_RPM * 2 * np.pi) / 60.0
Motor_Torque_Nm = np.round(np.clip((P_load_W / (omega + 1e-9)), -200, 500), 1)
motor_eff = 0.75 + 0.2 * np.exp(-((Motor_RPM-3000)/2500.0)**2) + 0.06 *
np.tanh(np.clip(Motor_Torque_Nm,0,300)/150.0)
motor_eff = np.clip(motor_eff, 0.6, 0.98).round(3)
```

```
# SOC per trip
battery_capacity_kWh = 60.0
soc = np.zeros(n)
start_idx = 0
for l in trip_lengths:
    soc_val = np.random.uniform(60,100)
    end_idx = start_idx + l
    energy = Theoretical_Energy_kWh[start_idx:end_idx]
    energy_share = energy * (1 + np.random.normal(0,0.05,size=l))
    soc_segment = soc_val - (np.cumsum(energy_share) / battery_capacity_kWh * 100.0)
    soc[start_idx:end_idx] = np.clip(soc_segment, 5, 100)
    start_idx = end_idx
State_of_Charge_pct = np.round(soc,2)
Battery_Voltage_V = np.round(np.clip(350 + 0.45*(Battery_Temperature_C-25) +
0.25*(State_of_Charge_pct-50) + np.random.normal(0,2,n), 320,420),1)
I = P_load_W / (Battery_Voltage_V + 1e-9)
regen_mask = Acceleration_ms2 < -0.2
I_regen = -0.6 * np.abs(I) * (np.minimum(1.0, np.abs(Acceleration_ms2)/3.0))
Battery_Current_A = np.round(np.where(regen_mask, I_regen, I),1)
Battery_Current_A = np.clip(Battery_Current_A, -400, 700)
Regen_Energy_kWh = np.round(np.where(Battery_Current_A<0, (-
Battery_Current_A*Battery_Voltage_V)/1000.0 * time_h, 0.0),4)

# Actual energy from refined physics + noise
E_refined = Theoretical_Energy_kWh / (motor_eff * (0.95 + 0.05*np.tanh((State_of_Charge_pct-30)/30)))
Energy_Consumption_kWh = np.round(np.clip(E_refined - 0.6*Regen_Energy_kWh +
np.random.normal(0,0.02,n), 0.0, 200.0),4)
Wind_Speed_mps = np.round(np.abs(np.random.normal(2.5 + 0.5*(Weather_Condition=='Rain'), 1.2,
n)).clip(0,12),2)
Tire_Pressure_PSI = np.round(np.random.uniform(30,36,n),1)
GPS_Elevation_m = np.round(np.clip(np.cumsum((Slope_pct/100.0) * Distance_Travelled_km *1000.0) %
2000, 0, 3000),1)

# Timestamps (30s segments) and driver id
timestamps = []
start_time = pd.Timestamp("2025-01-01 08:00:00")
```

```
for l in trip_lengths:
    for s in range(l):
        timestamps.append(start_time + pd.Timedelta(seconds=30*s))
    start_time += pd.Timedelta(minutes=5) + pd.Timedelta(seconds=30*l)
Driver_ID = np.random.choice([f"Driver_{i}" for i in range(1,21)], n)
df = pd.DataFrame({
    'Timestamp': timestamps[:n],
    'Trip_ID': trip_id[:n],
    'Driver_ID': Driver_ID,
    'Speed_kmh': Speed_kmh,
    'Acceleration_ms2': Acceleration_ms2,
    'Slope_%': Slope_pct,
    'Distance_Travelled_km': Distance_Travelled_km,
    'Vehicle_Weight_kg': Vehicle_Weight_kg,
    'Payload_kg': Payload_kg,
    'Battery_Temperature_C': Battery_Temperature_C,
    'Weather_Condition': Weather_Condition,
    'Traffic_Condition': Traffic_Condition,
    'Driving_Mode': Driving_Mode,
    'Road_Type': Road_Type,
    'Wind_Speed_mps': Wind_Speed_mps,
    'Tire_Pressure_PSI': Tire_Pressure_PSI,
    'GPS_Elevation_m': GPS_Elevation_m,
    'State_of_Charge_pct': State_of_Charge_pct,
    'Motor_RPM': Motor_RPM,
    'Motor_Torque_Nm': Motor_Torque_Nm,
    'Motor_Efficiency': motor_eff,
    'Battery_Voltage_V': Battery_Voltage_V,
    'Battery_Current_A': Battery_Current_A,
    'Regen_Energy_kWh': Regen_Energy_kWh,
    'Theoretical_Power_Demand': Theoretical_Power_Demand,
    'Theoretical_Energy_kWh': Theoretical_Energy_kWh,
    'Energy_Consumption_kWh': Energy_Consumption_kWh
})
df.to_csv(out_path, index=False)
print(f"Saved enhanced dataset to: {out_path}")
```

```
    return df

# Generate dataset (change n if you want larger)
df = generate_enhanced_dataset(out_path="EV_Physics_Enhanced_Dataset_v3.csv", n=2000)
df.head()

# show some stats
print(df.describe().round(3).T[['mean','std','min','max']].head(12))
print("\nColumns:", df.columns.tolist())
```

==========

BASIC TABNET vs PROPOSED PG-TABNET (FAST MODE)

==========

```
import numpy as np
import pandas as pd
import torch
from pytorch_tabnet.tab_model import TabNetRegressor
from pytorch_tabnet.metrics import Metric
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

DATA_PATH = "EV_Physics_Enhanced_Dataset_v3.csv"
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

MAX_EPOCHS = 80
PATIENCE = 20
BATCH_SIZE = 128
VIRTUAL_BATCH = 64
TEST_SIZE = 0.2
LAMBDA_PHYSICS = 0.002
SOC_WEIGHT = 0.2
```

```
==========
# Load Data
==========
df = pd.read_csv(DATA_PATH, parse_dates=['Timestamp'])
===============
# Feature Selection
===============
num_cols = [
    'Speed_kmh','Acceleration_ms2','Slope_%','Distance_Travelled_km',
    'Vehicle_Weight_kg','Payload_kg','Battery_Temperature_C',
    'Motor_RPM','Motor_Torque_Nm','Battery_Voltage_V',
    'Battery_Current_A','Regen_Energy_kWh',
    'Theoretical_Power_Demand','Theoretical_Energy_kWh'
]
num_cols = [c for c in num_cols if c in df.columns]
cat_cols = ['Weather_Condition','Traffic_Condition','Driving_Mode','Road_Type']
cat_cols = [c for c in cat_cols if c in df.columns]
========================
# Encode Categorical Features
========================
for c in cat_cols:
    df[c] = LabelEncoder().fit_transform(df[c].astype(str))
================
# Target Engineering
================
df['Energy_Residual_kWh'] = (
    df['Energy_Consumption_kWh'] - df['Theoretical_Energy_kWh']
)
df['SOC_Drop_pct'] = (df['Energy_Consumption_kWh'] / 60.0) * 100.0
=============
# Input Matrices
=============
X = df[num_cols + cat_cols].values
y_res = df['Energy_Residual_kWh'].values.reshape(-1, 1)
y_soc = df['SOC_Drop_pct'].values.reshape(-1, 1)
========================
```

 # Train / Test Split (Index Safe)

```
========================
idx_all = np.arange(len(df))
train_idx, test_idx = train_test_split(
    idx_all, test_size=TEST_SIZE, random_state=SEED
)
X_train = X[train_idx]
X_test  = X[test_idx]
y_res_train = y_res[train_idx]
y_res_test  = y_res[test_idx]
y_soc_train = y_soc[train_idx]
y_soc_test  = y_soc[test_idx]
theo_test = df.iloc[test_idx]['Theoretical_Energy_kWh'].values.reshape(-1,1)
actual_test = df.iloc[test_idx]['Energy_Consumption_kWh'].values.reshape(-1,1)
=======
```

 # Scaling

```
=======
scaler_X = StandardScaler()
X_train[:, :len(num_cols)] = scaler_X.fit_transform(X_train[:, :len(num_cols)])
X_test[:, :len(num_cols)]  = scaler_X.transform(X_test[:, :len(num_cols)])
res_scaler = StandardScaler()
soc_scaler = StandardScaler()
y_res_train_s = res_scaler.fit_transform(y_res_train)
y_res_test_s  = res_scaler.transform(y_res_test)
y_soc_train_s = soc_scaler.fit_transform(y_soc_train)
y_soc_test_s  = soc_scaler.transform(y_soc_test)
y_train_pg = np.hstack([y_res_train_s, y_soc_train_s])
y_test_pg  = np.hstack([y_res_test_s,  y_soc_test_s])
=================
```

 # Categorical Indexing

```
=================
cat_idxs = list(range(len(num_cols), len(num_cols) + len(cat_cols)))
cat_dims = [df[c].nunique() for c in cat_cols]
=======
```

 # Metric

```
=======
```

```
class ResidualRMSE(Metric):
    def __init__(self):
        self._name = "residual_rmse"
        self._maximize = False
    def __call__(self, y_true, y_score):
        return np.sqrt(mean_squared_error(y_true[:,0], y_score[:,0]))
```

==========

 # Physics-Guided Loss (Fast)

==========

```
class FastPhysicsGuidedLoss(torch.nn.Module):
    def __init__(self, lambda_reg=LAMBDA_PHYSICS, soc_weight=SOC_WEIGHT):
        super().__init__()
        self.lambda_reg = lambda_reg
        self.soc_weight = soc_weight
        self.mse = torch.nn.MSELoss()
    def forward(self, y_pred, y_true):
        res_pred = y_pred[:,0]
        soc_pred = y_pred[:,1]
        res_true = y_true[:,0]
        soc_true = y_true[:,1]
        data_loss = self.mse(res_pred, res_true)
        physics_loss = self.mse(res_pred, torch.zeros_like(res_pred))
        soc_loss = self.mse(soc_pred, soc_true)
        return data_loss + self.lambda_reg * physics_loss + self.soc_weight * soc_loss
```

==========

 # Basic Tabnet (Baseline)

==========

```
basic_tabnet = TabNetRegressor(
    n_d=16,
    n_a=16,
    n_steps=3,
    gamma=1.3,
    cat_idxs=cat_idxs,
    cat_dims=cat_dims,
    optimizer_fn=torch.optim.Adam,
    optimizer_params=dict(lr=0.01),
```

```
    verbose=1
)
print("\n🚀 Training BASIC TabNet...")
basic_tabnet.fit(
    X_train=X_train,
    y_train=y_res_train_s,
    eval_set=[(X_test, y_res_test_s)],
    eval_metric=[ResidualRMSE],
    max_epochs=60,
    patience=15,
    batch_size=BATCH_SIZE,
    virtual_batch_size=VIRTUAL_BATCH
)
basic_residual = res_scaler.inverse_transform(
    basic_tabnet.predict(X_test)
)
basic_energy_pred = theo_test + basic_residual
==========================
# Proposed Pg-Tabnet (Fast Mode)
==========================
pg_tabnet = TabNetRegressor(
    n_d=32,
    n_a=32,
    n_steps=4,
    gamma=1.5,
    cat_idxs=cat_idxs,
    cat_dims=cat_dims,
    optimizer_fn=torch.optim.Adam,
    optimizer_params=dict(lr=0.01),
    scheduler_fn=torch.optim.lr_scheduler.StepLR,
    scheduler_params={"step_size":25, "gamma":0.9},
    verbose=1
)
print("\n🚀 Training PROPOSED PG-TabNet...")
pg_tabnet.fit(
    X_train=X_train,
```

```
    y_train=y_train_pg,
    eval_set=[(X_test, y_test_pg)],
    eval_metric=[ResidualRMSE],
    max_epochs=MAX_EPOCHS,
    patience=PATIENCE,
    batch_size=BATCH_SIZE,
    virtual_batch_size=VIRTUAL_BATCH,
    loss_fn=FastPhysicsGuidedLoss()
)
pg_pred_scaled = pg_tabnet.predict(X_test)
pg_residual = res_scaler.inverse_transform(
    pg_pred_scaled[:,0].reshape(-1,1)
)
pg_energy_pred = theo_test + pg_residual
```

==========

 # Final Comparison

==========

```
def evaluate(name, y_true, y_pred):
    print(
        f"[{name}] RMSE:",
        np.sqrt(mean_squared_error(y_true, y_pred)),
        "| R2:",
        r2_score(y_true, y_pred)
    )
print("\n📊 FINAL MODEL COMPARISON")
evaluate("Basic TabNet", actual_test, basic_energy_pred)
evaluate("Proposed PG-TabNet", actual_test, pg_energy_pred)
```

==========

 # Visualization

==========

```
idx = np.argsort(actual_test.flatten())[:100]
plt.figure(figsize=(13,5))
plt.plot(actual_test[idx], label="Actual", marker='o')
plt.plot(basic_energy_pred[idx], label="Basic TabNet", linestyle='--')
plt.plot(pg_energy_pred[idx], label="Proposed PG-TabNet", linestyle='-.')
plt.title("Basic TabNet vs Proposed PG-TabNet")
```

```
plt.ylabel("Energy (kWh)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

idx = np.argsort(actual_test.flatten())
plt.figure(figsize=(12,5))
plt.plot(basic_errors[idx], label="Basic TabNet Residual", linestyle='--')
plt.plot(pg_errors[idx], label="Proposed PG-TabNet Residual", linestyle='-')
plt.axhline(0, color='black', linewidth=1)
plt.title("Residual Comparison (Sorted Samples)")
plt.ylabel("Residual (kWh)")
plt.xlabel("Sample Index")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

plt.figure(figsize=(6,6))
plt.scatter(actual_test, basic_energy_pred, alpha=0.4, label='Basic TabNet')
plt.scatter(actual_test, pg_energy_pred, alpha=0.4, label='Proposed PG-TabNet')
max_val = actual_test.max()
plt.plot([0, max_val], [0, max_val], 'k--', label='Perfect Prediction')
plt.xlabel("Actual Energy (kWh)")
plt.ylabel("Predicted Energy (kWh)")
plt.title("Actual vs Predicted Energy")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
# Identify numerical columns
numerical_cols = df.select_dtypes(include=np.number).columns.tolist()
# Calculate the correlation matrix for numerical features
correlation_matrix = df[numerical_cols].corr()
```

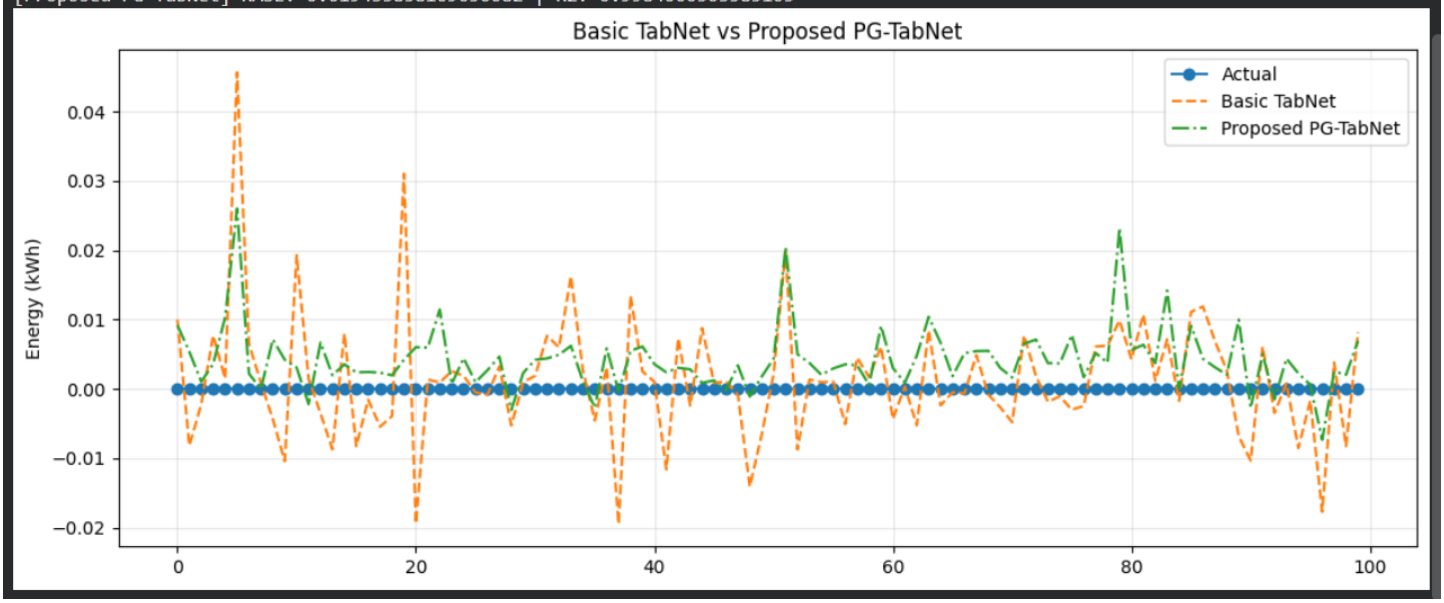
```
# Create a heatmap of the correlation matrix
plt.figure(figsize=(16, 12))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Correlation Heatmap of Numerical Features', fontsize=18)
plt.xticks(rotation=90)
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

APPENDIX B

SCREENSHOTS

```
/usr/local/lib/python3.12/dist-packages/pytorch_tabnet/abstract_model.py:82: UserWarning: Device used : cpu
  warnings.warn(f"Device used : {self.device}")

🚀 Training BASIC TabNet...
epoch 0  | loss: 1.8702  | val_0_residual_rmse: 1.3575  |  0:00:01s
epoch 1  | loss: 0.60819 | val_0_residual_rmse: 0.96509 |  0:00:03s
epoch 2  | loss: 0.50135 | val_0_residual_rmse: 0.87975 |  0:00:04s
epoch 3  | loss: 0.38946 | val_0_residual_rmse: 0.65159 |  0:00:04s
epoch 4  | loss: 0.41605 | val_0_residual_rmse: 0.5844  |  0:00:05s
epoch 5  | loss: 0.32025 | val_0_residual_rmse: 0.71019 |  0:00:05s
epoch 6  | loss: 0.28404 | val_0_residual_rmse: 0.57133 |  0:00:06s
epoch 7  | loss: 0.28723 | val_0_residual_rmse: 0.762   |  0:00:06s
epoch 8  | loss: 0.32304 | val_0_residual_rmse: 0.63924 |  0:00:07s
epoch 9  | loss: 0.2614  | val_0_residual_rmse: 0.79772 |  0:00:08s
epoch 10 | loss: 0.25908 | val_0_residual_rmse: 0.77561 |  0:00:08s
epoch 11 | loss: 0.2328  | val_0_residual_rmse: 0.48442 |  0:00:09s
epoch 12 | loss: 0.25286 | val_0_residual_rmse: 0.43099 |  0:00:10s
epoch 13 | loss: 0.23035 | val_0_residual_rmse: 0.41406 |  0:00:10s
epoch 14 | loss: 0.22618 | val_0_residual_rmse: 0.49843 |  0:00:11s
epoch 15 | loss: 0.20365 | val_0_residual_rmse: 0.51517 |  0:00:12s
epoch 16 | loss: 0.22464 | val_0_residual_rmse: 0.50836 |  0:00:12s
epoch 17 | loss: 0.2012  | val_0_residual_rmse: 0.48858 |  0:00:13s
epoch 18 | loss: 0.23576 | val_0_residual_rmse: 0.47532 |  0:00:13s
epoch 19 | loss: 0.1833  | val_0_residual_rmse: 0.51687 |  0:00:14s
epoch 20 | loss: 0.2156  | val_0_residual_rmse: 0.46149 |  0:00:14s
epoch 21 | loss: 0.18956 | val_0_residual_rmse: 0.50816 |  0:00:15s
epoch 22 | loss: 0.14929 | val_0_residual_rmse: 0.53608 |  0:00:15s
epoch 23 | loss: 0.17548 | val_0_residual_rmse: 0.47586 |  0:00:15s
epoch 24 | loss: 0.1698  | val_0_residual_rmse: 0.48429 |  0:00:16s
epoch 25 | loss: 0.21362 | val_0_residual_rmse: 0.52058 |  0:00:16s
epoch 26 | loss: 0.17252 | val_0_residual_rmse: 0.52314 |  0:00:16s
epoch 27 | loss: 0.16123 | val_0_residual_rmse: 0.61667 |  0:00:17s
epoch 28 | loss: 0.17613 | val_0_residual_rmse: 0.50074 |  0:00:17s

Early stopping occurred at epoch 28 with best_epoch = 13 and best_val_0_residual_rmse = 0.41406

🚀 Training PROPOSED PG-TabNet...
/usr/local/lib/python3.12/dist-packages/pytorch_tabnet/callbacks.py:172: UserWarning: Best weights from best epoch are automatically used!
  warnings.warn(wrn_msg)
/usr/local/lib/python3.12/dist-packages/pytorch_tabnet/abstract_model.py:82: UserWarning: Device used : cpu
  warnings.warn(f"Device used : {self.device}")
epoch 0  | loss: 3.18512 | val_0_residual_rmse: 1.5961  |  0:00:00s
epoch 1  | loss: 1.00976 | val_0_residual_rmse: 0.85054 |  0:00:00s
epoch 2  | loss: 0.71876 | val_0_residual_rmse: 0.9298  |  0:00:01s
epoch 3  | loss: 0.51041 | val_0_residual_rmse: 0.77107 |  0:00:01s
epoch 4  | loss: 0.47527 | val_0_residual_rmse: 0.85357 |  0:00:02s
epoch 5  | loss: 0.42524 | val_0_residual_rmse: 0.62103 |  0:00:02s
epoch 6  | loss: 0.32779 | val_0_residual_rmse: 0.57436 |  0:00:02s
epoch 7  | loss: 0.26744 | val_0_residual_rmse: 0.49633 |  0:00:03s
epoch 8  | loss: 0.28837 | val_0_residual_rmse: 0.53743 |  0:00:03s
epoch 9  | loss: 0.2854  | val_0_residual_rmse: 0.64736 |  0:00:04s
epoch 10 | loss: 0.26223 | val_0_residual_rmse: 0.54248 |  0:00:04s
epoch 11 | loss: 0.27457 | val_0_residual_rmse: 0.6045  |  0:00:05s
epoch 12 | loss: 0.27458 | val_0_residual_rmse: 0.58769 |  0:00:05s
epoch 13 | loss: 0.27427 | val_0_residual_rmse: 0.65084 |  0:00:05s
epoch 14 | loss: 0.27597 | val_0_residual_rmse: 0.56067 |  0:00:06s
epoch 15 | loss: 0.27348 | val_0_residual_rmse: 0.59227 |  0:00:06s
epoch 16 | loss: 0.24306 | val_0_residual_rmse: 0.56242 |  0:00:07s
epoch 17 | loss: 0.23273 | val_0_residual_rmse: 0.63218 |  0:00:07s
epoch 18 | loss: 0.25609 | val_0_residual_rmse: 0.56138 |  0:00:07s
epoch 19 | loss: 0.21213 | val_0_residual_rmse: 0.59333 |  0:00:08s
epoch 20 | loss: 0.1979  | val_0_residual_rmse: 0.50307 |  0:00:09s
epoch 21 | loss: 0.17334 | val_0_residual_rmse: 0.61127 |  0:00:09s
epoch 22 | loss: 0.22217 | val_0_residual_rmse: 0.60397 |  0:00:10s
epoch 23 | loss: 0.18277 | val_0_residual_rmse: 0.47317 |  0:00:10s
epoch 24 | loss: 0.23808 | val_0_residual_rmse: 0.46668 |  0:00:11s
epoch 25 | loss: 0.2254  | val_0_residual_rmse: 0.46751 |  0:00:11s
epoch 26 | loss: 0.20924 | val_0_residual_rmse: 0.49799 |  0:00:12s
epoch 27 | loss: 0.20556 | val_0_residual_rmse: 0.45406 |  0:00:12s
epoch 28 | loss: 0.18583 | val_0_residual_rmse: 0.49027 |  0:00:13s
epoch 29 | loss: 0.17476 | val_0_residual_rmse: 0.45786 |  0:00:13s
epoch 30 | loss: 0.20091 | val_0_residual_rmse: 0.41087 |  0:00:13s
epoch 31 | loss: 0.14312 | val_0_residual_rmse: 0.46593 |  0:00:14s
epoch 32 | loss: 0.15491 | val_0_residual_rmse: 0.44438 |  0:00:14s
epoch 33 | loss: 0.17742 | val_0_residual_rmse: 0.41481 |  0:00:15s
epoch 34 | loss: 0.16234 | val_0_residual_rmse: 0.42809 |  0:00:15s
epoch 35 | loss: 0.19275 | val_0_residual_rmse: 0.35409 |  0:00:16s
epoch 36 | loss: 0.16705 | val_0_residual_rmse: 0.37719 |  0:00:16s
epoch 37 | loss: 0.15109 | val_0_residual_rmse: 0.36437 |  0:00:16s
epoch 38 | loss: 0.16066 | val_0_residual_rmse: 0.61164 |  0:00:17s
epoch 39 | loss: 0.18691 | val_0_residual_rmse: 0.44465 |  0:00:17s
epoch 40 | loss: 0.1793  | val_0_residual_rmse: 0.45304 |  0:00:18s
epoch 41 | loss: 0.17141 | val_0_residual_rmse: 0.4262  |  0:00:18s
epoch 42 | loss: 0.14554 | val_0_residual_rmse: 0.39803 |  0:00:19s
epoch 43 | loss: 0.15491 | val_0_residual_rmse: 0.37509 |  0:00:19s
epoch 44 | loss: 0.15979 | val_0_residual_rmse: 0.34615 |  0:00:19s
epoch 45 | loss: 0.15416 | val_0_residual_rmse: 0.36621 |  0:00:20s
epoch 46 | loss: 0.14786 | val_0_residual_rmse: 0.47485 |  0:00:20s
epoch 47 | loss: 0.16733 | val_0_residual_rmse: 0.41026 |  0:00:21s
epoch 48 | loss: 0.25103 | val_0_residual_rmse: 0.4658  |  0:00:22s
epoch 49 | loss: 0.17301 | val_0_residual_rmse: 0.53382 |  0:00:22s
epoch 50 | loss: 0.16063 | val_0_residual_rmse: 0.41972 |  0:00:23s
epoch 51 | loss: 0.22156 | val_0_residual_rmse: 0.49603 |  0:00:23s
epoch 52 | loss: 0.14731 | val_0_residual_rmse: 0.46745 |  0:00:24s
epoch 53 | loss: 0.14136 | val_0_residual_rmse: 0.39484 |  0:00:24s
epoch 54 | loss: 0.13541 | val_0_residual_rmse: 0.36678 |  0:00:25s
epoch 55 | loss: 0.14974 | val_0_residual_rmse: 0.45389 |  0:00:25s
epoch 56 | loss: 0.14537 | val_0_residual_rmse: 0.39492 |  0:00:25s
epoch 57 | loss: 0.1401  | val_0_residual_rmse: 0.45239 |  0:00:26s
epoch 58 | loss: 0.17394 | val_0_residual_rmse: 0.41318 |  0:00:26s
epoch 59 | loss: 0.14621 | val_0_residual_rmse: 0.46393 |  0:00:27s
epoch 60 | loss: 0.15012 | val_0_residual_rmse: 0.42286 |  0:00:27s
epoch 61 | loss: 0.15442 | val_0_residual_rmse: 0.5812  |  0:00:28s
epoch 62 | loss: 0.17193 | val_0_residual_rmse: 0.48024 |  0:00:28s
epoch 63 | loss: 0.20457 | val_0_residual_rmse: 0.38401 |  0:00:28s
epoch 64 | loss: 0.13016 | val_0_residual_rmse: 0.44189 |  0:00:29s

Early stopping occurred at epoch 64 with best_epoch = 44 and best_val_0_residual_rmse = 0.34615
/usr/local/lib/python3.12/dist-packages/pytorch_tabnet/callbacks.py:172: UserWarning: Best weights from best epoch are automatically used!
  warnings.warn(wrn_msg)
```

```
📊 FINAL MODEL COMPARISON
[Basic TabNet] RMSE: 0.023246259719925326 | R2: 0.9977107236067783
[Proposed PG-TabNet] RMSE: 0.019433858109056682 | R2: 0.9984000363585103
```




```
📊 Basic TabNet Accuracy
RMSE  : 0.02325 kWh
R²    : 0.99771
MAE   : 0.01466 kWh
MAPE  : 214918194.10 %

📊 Proposed PG-TabNet Accuracy
RMSE  : 0.01943 kWh
R²    : 0.99840
MAE   : 0.01268 kWh
MAPE  : 160038324.58 %
```

	Model	RMSE (kWh)	R²	MAE (kWh)	MAPE (%)
0	Basic TabNet	0.023246	0.997711	0.014662	2.149182e+08
1	Proposed PG-TabNet	0.019434	0.998400	0.012682	1.600383e+08

REFERENCES

[1] S. Karpatne, W. Watkins, J. Read, and V. Kumar, "Physics-guided neural networks (PGNN): An application in lake temperature modeling," *IEEE Transactions on Knowledge and Data Engineering*, vol. 29, no. 12, pp. 2581–2594, Dec. 2017.

[2] A. Arik and T. Pfister, "TabNet: Attentive interpretable tabular learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, pp. 6679–6687, 2021.

[4] M. Hussain, M. A. Khan, and S. Lee, "Machine learning-based energy consumption prediction for electric vehicles using real-world driving data," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 5, pp. 4567–4578, May 2023.

[5] J. Zhu, Y. Chen, and X. Li, "Data-driven electric vehicle energy consumption prediction considering road gradient and traffic conditions," *Applied Energy*, vol. 356, Art. no. 121010, 2024.

[6] A. Akshay, P. K. Singh, and R. Kumar, "Hybrid ensemble learning for electric vehicle energy consumption forecasting," *Energy*, vol. 287, Art. no. 129226, 2024.

[7] R. Saldarini, A. Brambilla, and M. Fiori, "Impact of road elevation and driving behavior on electric vehicle energy consumption," *Energy Conversion and Management*, vol. 295, Art. no. 117704, 2024.

[8] Y. Li, H. Zhang, and Z. Wang, "Robust prediction of electric vehicle energy consumption using gradient boosting methods," *Energy Reports*, vol. 10, pp. 221–233, 2024.

[9] H. Chen, L. Xu, and M. Zhao, "Physics-informed machine learning for vehicle energy consumption prediction," *Energy and AI*, vol. 15, Art. no. 100302, 2024.

[10] P. Sahoo and S. Debnath, "A comparative analysis of regression and ensemble models for electric vehicle energy estimation," *International Journal of Energy Research*, vol. 45, no. 12, pp. 17892–17907, 2021.

[11] S. K. Behera, D. K. Panda, and S. Das, "Sequential deep learning framework for electric vehicle energy prediction," *IEEE Access*, vol. 11, pp. 45678–45690, 2023.

[12] X. Liu, J. Sun, and Y. Zhou, "Uncertainty-aware energy consumption forecasting for electric vehicles using probabilistic learning," *Applied Energy*, vol. 365, Art. no. 121245, 2024.

[13] A. Kumar, S. Verma, and N. Patel, "Feature engineering strategies for electric vehicle energy prediction using large-scale datasets," *Symmetry*, vol. 17, no. 6, Art. no. 930, 2025.

[14] L. Wang, Z. Yu, and Q. Chen, "Hybrid physics–data-driven models for electric vehicle energy estimation," *Energy*, vol. 292, Art. no. 129876, 2025.

[15] T. Nguyen, H. Pham, and K. Tran, "Energy consumption analysis of electric vehicles considering environmental factors," *Energy Conversion and Management*, vol. 304, Art. no. 118205, 2025.

[16] H. Huang, H. He, Y. Wang, Z. Zhang, and T. Wang, "Energy consumption prediction of electric vehicles for data-scarce scenarios using a large-scale pre-trained model," Transportation Research Part D: Transport and Environment, vol. 112, Art. no. 104830, 2025.

[17] S. Zhang, Y. Li, and H. Liu, “Physics-guided deep learning for transportation energy systems,” *Scientific Reports*, vol. 15, Art. no. 14129, 2025.

[18] R. Mehta and K. Shah, “Deep learning-based state-of-charge estimation for lithium-ion batteries in electric vehicles,” *Energy Reports*, vol. 11, pp. 1456–1468, 2025.

[19] J. Park, M. Kim, and D. Lee, “Data-driven modeling of electric vehicle energy consumption under diverse driving conditions,” *Journal of Cleaner Production*, vol. 432, Art. no. 140112, 2025.

[20] A. Khalid, M. Ali, and S. Rehman, “Review of machine learning approaches for electric vehicle energy consumption prediction,” *International Journal of Sustainable Energy*, vol. 41, no. 6, pp. 987–1005, 2022.

[21] Y. Liu, Z. Chen, and X. Zhang, “Physics-informed deep learning for electric vehicle energy consumption prediction under varying driving conditions,” Applied Energy, vol. 362, Art. no. 121132, 2024.

[22] M. R. Islam, J. Kim, and S. Park, “Robust energy consumption modeling of electric vehicles using hybrid physics–machine learning approaches,” Energy Conversion and Management, vol. 298, Art. no. 117987, 2024.

[23] A. Verma, R. Singh, and P. Gupta, “Attention-based deep learning framework for electric vehicle energy estimation using tabular driving data,” IEEE Access, vol. 12, pp. 145321–145334, 2024.

[24] H. Zhao, L. Wang, and Y. Sun, “Uncertainty-aware energy consumption prediction for electric vehicles using physics-regularized neural networks,” Energy and AI, vol. 16, Art. no. 100345, 2025.

[25] S. Morales, D. Rojas, and F. Pérez, “Residual learning-based hybrid modeling for electric vehicle energy forecasting in real-world scenarios,” Journal of Cleaner Production, vol. 438, Art. no. 140785, 2025

[26] EV_Physics_Enhanced_Dataset_v3.